Netflix: Data Exploration & Visualisation

What should Netflix produce next, and where should it grow?

Author: Guntuku Hariprasad | **Dataset:** 8,807 Netflix titles (movies & TV shows), catalogue snapshot up to 25 Sep 2021 **Tools:** Python, pandas, NumPy, Matplotlib, Seaborn

---

Executive summary (for business leaders)

What we found	Number behind it
The catalogue is **movie-heavy**, but the mix is shifting towards TV	70% movies / 30% TV overall; TV share of new additions rose from **25% (2018) to 34% (2021)**
For the newest releases, **TV now beats movies**	Titles released in 2021: **315 TV shows vs 277 movies**
Content growth **peaked in 2019** and slowed during 2020–21	2,016 titles added in 2019 → 1,879 in 2020
Netflix is an **adult-first** library	45.6% of titles are rated for adults (TV-MA / R / NC-17); only 6.5% are made for little kids
Three markets supply 59% of the catalogue	United States (3,690), India (1,046), United Kingdom (806)
Each country has a **distinct content "personality"**	India = 92% movies (dramas, Bollywood stars); Japan & South Korea = majority TV (anime, K-dramas)
Fast-rising newcomer markets	Nigeria (1 → 42 titles/yr, 2018→2020), Egypt (3 → 54); Japan & South Korea up 70–80%
Most TV shows **never get past season 1**	67% of TV shows have exactly 1 season
The **sweet-spot movie is ~1.5–2 hours**	Half of all movies run 87–114 min (median 98 min)
Friday and the **1st of the month** are the launch days	28% of titles added on a Friday; 2,212 titles added on the 1st

Top recommendations (detailed in Section 13): invest more in original TV series (especially international dramas, crime and kids' shows); build local-language content hubs in India, Japan, South Korea and fast-growing markets like Nigeria and Egypt; keep films around 90–110 minutes; release big shows on Fridays in **July or December**; and grow the family/kids slate, which is under-represented today.

Table of contents

1. Problem statement & approach
2. Setup & data loading
3. Basic metrics: shape, data types, sample
4. Data cleaning & feature engineering (incl. conversion to `category`)
5. Missing value detection & treatment
6. Statistical summary & range of attributes
7. Non-graphical analysis: value counts & unique values
8. Pre-processing: un-nesting cast, director, country & genre
9. Univariate visual analysis
10. Bivariate & multivariate visual analysis
11. Outlier check

12. Business insights
13. Recommendations

1. Problem statement & approach

Business problem. Netflix (222M+ subscribers) must decide **which type of shows and movies to produce or license** and **how to grow in different countries**. Every content rupee/dollar is a bet; this analysis turns the current catalogue into evidence for those bets.

Questions this study answers

1. What is the movie vs TV-show mix, and is Netflix focusing more on TV in recent years?
2. How has the number of titles released/added changed over the last 20–30 years?
3. What genres, ratings (audiences) and durations dominate, and what is growing?
4. What content is available in different countries, and which markets are rising?
5. Who are the most frequent actors and directors by content type and market?
6. What is the best time (month / day) to launch a TV show or movie?

Approach. Clean the data → profile it (shape, types, missing values, statistics) → un-nest multi-value columns (cast, director, country, genre) → univariate & bivariate visuals → outlier checks → convert patterns into business insights and plain-language recommendations.

> **Important caveat:** the data describes what is *on* Netflix, not what people *watch*. Viewing hours, ratings and revenue are not available, so "more titles" means "more investment/supply", not necessarily "more demand". 2021 only covers January to 25 September.

2. Setup & data loading

```
import warnings
warnings.filterwarnings('ignore')
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns

pd.set_option('display.max_columns', 50)
pd.set_option('display.width', 160)

# ---- consistent, accessible chart style (colour-blind safe pair validated) ----
MOVIE, TV, INK, MUTED = '#e34948', '#2a78d6', '#2b2b2b', '#8a8a86'
PAL = {'Movie': MOVIE, 'TV Show': TV}
sns.set_theme(style='whitegrid', rc={
    'figure.dpi': 110, 'savefig.dpi': 150, 'axes.edgecolor': '#d0d0cc',
'grid.color': '#ececea',
    'axes.titleweight': 'bold', 'axes.titlesize': 12, 'axes.labelcolor': INK,
'text.color': INK,
    'xtick.color': '#52514e', 'ytick.color': '#52514e', 'font.size': 10})
```

```
IMG = Path('../images'); IMG.mkdir(exist_ok=True)
def save(name):
    'Save the current figure into /images for the README / portfolio.'
    plt.tight_layout()
    plt.savefig(IMG / f'{name}.png', bbox_inches='tight', facecolor='white')
    plt.show()

def label_bars(ax, fmt='{:,.0f}', fontsize=8):
    for c in ax.containers:
        ax.bar_label(c, fmt=fmt, fontsize=fontsize, padding=2, color='#52514e')
```

```
raw = pd.read_csv('../data/netflix.csv')
df = raw.copy()
df.head()
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	descrip
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	A fa near end c life, filr
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, TV Dramas, TV Mysteries	cros paths par Cape T
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021	TV-MA	1 Season	Crime TV Shows, International TV Shows, TV Act...	To pr his fa fr pow drug
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021	TV-MA	1 Season	Docuseries, Reality TV	Fe flirta and ta c a
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, Romantic TV Shows, TV ...	In a ci coac cer knov tra

3. Basic metrics: shape, data types & sample

```
print(f'Rows: {df.shape[0]:,}  |  Columns: {df.shape[1]}')
print(f'Duplicate rows: {df.duplicated().sum()}  |  Duplicate show_id:
{df.show_id.duplicated().sum()}  |  Duplicate titles:
{df.title.duplicated().sum()}')
df.info()
```

```
Rows: 8,807  |  Columns: 12
Duplicate rows: 0  |  Duplicate show_id: 0  |  Duplicate titles: 0
<class 'pandas.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   show_id       8807 non-null   str
 1   type          8807 non-null   str
 2   title         8807 non-null   str
 3   director      6173 non-null   str
 4   cast          7982 non-null   str
 5   country       7976 non-null   str
 6   date_added    8797 non-null   str
 7   release_year  8807 non-null   int64
 8   rating        8803 non-null   str
 9   duration      8804 non-null   str
 10  listed_in     8807 non-null   str
 11  description   8807 non-null   str
dtypes: int64(1), str(11)
memory usage: 825.8 KB
```

Observations

- The dataset has **8,807 rows × 12 columns**; each row is one unique title (no duplicate rows or IDs). After trimming stray spaces, one title (*Consequences*, a 2014 Turkish movie) appears twice, which is a negligible duplicate listing.
- Only `release_year` is numeric (`int64`). Everything else is text (`object` / `str`), including `date_added` (should be a date) and `duration` (mixes "90 min" and "2 Seasons", so it needs splitting into a number and a unit).
- `cast`, `director`, `country` and `listed_in` hold **comma-separated lists** (e.g. several actors per title), so they must be **un-nested** before counting.

4. Data cleaning & feature engineering

4.1 Fixing a data-entry error. Three Louis C.K. stand-up specials have their duration typed into the `rating` column.

```
bad = df['rating'].str.contains('min', na=False)
display(df.loc[bad, ['title', 'type', 'rating', 'duration']])
df.loc[bad, 'duration'] = df.loc[bad, 'rating']   # move the value to the right
column
df.loc[bad, 'rating'] = np.nan                    # the true rating is unknown
```

	title	type	rating	duration
5541	Louis C.K. 2017	Movie	74 min	NaN
5794	Louis C.K.: Hilarious	Movie	84 min	NaN
5813	Louis C.K.: Live at the Comedy Store	Movie	66 min	NaN

4.2 Trimming text, parsing dates and splitting duration into a number + unit; creating time and audience features.

```
for col in ['type', 'title', 'director', 'cast', 'country', 'date_added', 'rating',
'duration', 'listed_in']:
    df[col] = df[col].str.strip()

df['date_added']   = pd.to_datetime(df['date_added'], format='%B %d, %Y',
errors='coerce')
df['year_added']   = df['date_added'].dt.year.astype('Int64')
df['month_added']  = df['date_added'].dt.month.astype('Int64')
df['month_name']   = df['date_added'].dt.month_name().str[:3]
df['weekday_added']= df['date_added'].dt.day_name()
df['day_added']    = df['date_added'].dt.day.astype('Int64')

df['duration_num']  = df['duration'].str.extract(r'(\d+)').astype(float)
df['duration_unit'] = np.where(df['duration'].str.contains('Season', na=False),
'Seasons', 'Minutes')

# How many years after release did the title reach Netflix?
df['years_to_netflix'] = df['year_added'] - df['release_year']

# Audience group: maps 14 rating codes to 4 groups a business reader understands
audience = {'TV-Y': 'Little Kids', 'TV-G': 'Little Kids', 'G': 'Little Kids',
            'TV-Y7': 'Older Kids', 'TV-Y7-FV': 'Older Kids', 'PG': 'Older Kids',
'TV-PG': 'Older Kids',
            'PG-13': 'Teens', 'TV-14': 'Teens',
            'R': 'Adults', 'NC-17': 'Adults', 'TV-MA': 'Adults',
            'NR': 'Unrated', 'UR': 'Unrated'}
df['audience'] = df['rating'].map(audience)

# Release era for coarse comparisons
df['release_era'] = pd.cut(df['release_year'], [1900, 1989, 1999, 2009, 2015, 2021],
                           labels=['Before 1990', '1990s', '2000s', '2010–15',
'2016–21'])
```

4.3 Converting categorical attributes to the `category` dtype (saves memory and fixes a logical order for ratings and audiences).

```
before = df.memory_usage(deep=True).sum() / 1024**2
rating_order = ['TV-Y', 'TV-Y7', 'TV-Y7-FV', 'TV-G', 'G', 'TV-PG', 'PG', 'PG-13',
'TV-14', 'R', 'TV-MA', 'NC-17', 'NR', 'UR']
df['type']          = df['type'].astype('category')
df['rating']        = pd.Categorical(df['rating'], categories=rating_order,
ordered=True)
df['audience']      = pd.Categorical(df['audience'], categories=['Little Kids',
'Older Kids', 'Teens', 'Adults', 'Unrated'], ordered=True)
df['duration_unit'] = df['duration_unit'].astype('category')
df['weekday_added'] = pd.Categorical(df['weekday_added'], categories=
['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday'],
ordered=True)
after = df.memory_usage(deep=True).sum() / 1024**2
print(f'Memory: {before:.2f} MB -> {after:.2f} MB')
df.dtypes
```

```
Memory: 9.74 MB -> 7.47 MB
```

```
show_id                        str
type                      category
title                          str
director                       str
cast                           str
country                        str
date_added          datetime64[us]
release_year                 int64
rating                    category
duration                       str
listed_in                      str
description                    str
year_added                   Int64
month_added                  Int64
month_name                     str
weekday_added             category
day_added                    Int64
duration_num               float64
duration_unit             category
years_to_netflix             Int64
audience                  category
release_era               category
dtype: object
```

Observations

- `type`, `rating`, `audience`, `duration_unit` and `weekday_added` are now `category` (ordered where order matters, e.g. ratings go from youngest to most mature audience).
- `date_added` is a real date, so we can analyse trends by year, month and weekday.
- `duration` is split into `duration_num` (minutes for movies, seasons for TV) + `duration_unit`. Movies and TV are therefore analysed **separately** for duration; mixing minutes and seasons would be meaningless.
- New features: `year_added`, `month_added`, `weekday_added`, `years_to_netflix`, `audience`, `release_era`.

5. Missing value detection & treatment

```
mv = pd.DataFrame({'missing': raw.isna().sum(), 'missing_after_fix':
df[raw.columns].isna().sum()})
mv['%'] = (mv['missing_after_fix'] / len(df) * 100).round(2)
mv = mv.sort_values('%', ascending=False); display(mv)

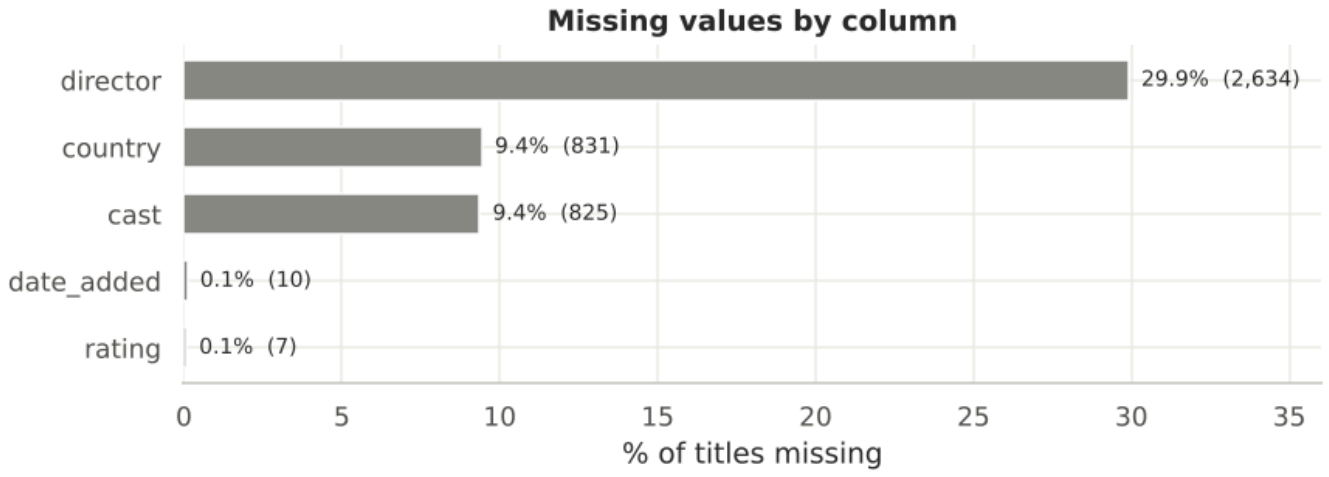
print('\nShare of missing values by content type (%):')
display((df.groupby('type', observed=True)[['director', 'cast', 'country',
'date_added', 'rating']]
         .agg(lambda s: s.isna().mean() * 100).round(1)))
```

	missing	missing_after_fix	%
director	2634	2634	29.91
country	831	831	9.44
cast	825	825	9.37
date_added	10	10	0.11
rating	4	7	0.08
title	0	0	0.00
show_id	0	0	0.00
type	0	0	0.00
release_year	0	0	0.00
duration	3	0	0.00
listed_in	0	0	0.00
description	0	0	0.00

```
Share of missing values by content type (%):
```

type	director	cast	country	date_added	rating
Movie	3.1	7.7	7.2	0.0	0.1
TV Show	91.4	13.1	14.6	0.4	0.1

```
fig, ax = plt.subplots(figsize=(8, 3))
m = mv[mv['%'] > 0]
ax.barh(m.index[::-1], m['%'][::-1], color=MUTED, height=0.6)
for i, (v, n) in enumerate(zip(m['%'][::-1], m['missing_after_fix'][::-1])):
    ax.text(v + 0.4, i, f'{v:.1f}%  ({n:,})', va='center', fontsize=9)
ax.set(title='Missing values by column', xlabel='% of titles missing', xlim=(0, 36))
sns.despine(left=True); save('01_missing_values')
```



Observations

- `director` **is the biggest gap (29.9%)**, but this is mostly structural: **91% of TV shows have no director** (TV is directed per episode), versus only 3% of movies.
- `cast` (9.4%) and `country` (9.4%) are moderately missing; TV shows are missing them about twice as often as movies.
- `date_added` (10 rows, all TV shows), `rating` (7) and `duration` (0 after the fix) are negligible.

Treatment (no rows dropped, so no information is lost):

- `director`, `cast`, `country` → filled with **"Unknown"** for value counts; "Unknown" is excluded from "Top-N" rankings so it does not distort them.
- `rating` → "Unknown" isn't a rating code, so the 7 rows are left as missing (category NaN) and simply ignored in rating plots.
- `date_added` → 10 rows left missing and ignored in time-based charts (0.1% of data).

```
for col in ['director', 'cast', 'country']:
    df[col] = df[col].fillna('Unknown')
df[['director', 'cast', 'country']].isna().sum()
```

```
director    0
cast        0
country     0
dtype: int64
```

6. Statistical summary & range of attributes

```
num = df[['release_year', 'year_added',
'years_to_netflix']].astype(float).describe().T
display(num.round(2))
print('Movies — duration in minutes:'); display(df.loc[df.type == 'Movie',
'duration_num'].describe().to_frame().T.round(1))
print('TV shows — number of seasons:'); display(df.loc[df.type == 'TV Show',
'duration_num'].describe().to_frame().T.round(2))
```

	count	mean	std	min	25%	50%	75%	max
release_year	8807.0	2014.18	8.82	1925.0	2013.0	2017.0	2019.0	2021.0
year_added	8797.0	2018.87	1.57	2008.0	2018.0	2019.0	2020.0	2021.0
years_to_netflix	8797.0	4.69	8.79	-3.0	0.0	1.0	5.0	93.0

Movies — duration in minutes:

	count	mean	std	min	25%	50%	75%	max
duration_num	6131.0	99.6	28.3	3.0	87.0	98.0	114.0	312.0

TV shows — number of seasons:

	count	mean	std	min	25%	50%	75%	max
duration_num	2676.0	1.76	1.58	1.0	1.0	1.0	2.0	17.0

```
df.describe(include=['object', 'category']).T
```

	count	unique	top	freq
show_id	8807	8807	s1	1
type	8807	2	Movie	6131
title	8807	8806	Consequences	2
director	8807	4529	Unknown	2634
cast	8807	7693	Unknown	825
country	8807	749	United States	2818
rating	8800	14	TV-MA	3207
duration	8807	220	1 Season	1793
listed_in	8807	514	Dramas, International Movies	362
description	8807	8775	Paranormal activity at a lush, abandoned prope...	4
month_name	8797	12	Jul	827
weekday_added	8797	7	Friday	2498
duration_unit	8807	2	Minutes	6131
audience	8800	5	Adults	4009
release_era	8807	5	2016–21	5656

6.1 Comments on the range of attributes

- `release_year` **spans 1925–2021 (96 years)** but is heavily concentrated in recent years: median **2017**, the middle 50% fall in **2013–2019**, and **84.8%** of titles were released in 2010 or later. Netflix is a *recent-content* platform; the classic catalogue is thin.
- `date_added` **spans Jan 2008 – 25 Sep 2021**, but meaningful volumes only start in **2015–16**, the years of Netflix's global expansion.
- **Movie duration ranges from 3 to 312 minutes**; mean 99.6 vs median 98, so the distribution is close to symmetric with long tails on both ends (shorts and epics).
- **TV seasons range from 1 to 17**, but the median is **1** and the mean only 1.76, i.e. strongly right-skewed.
- `years_to_netflix` ranges from −3 to 93 years. Negative values (14 titles) are data-entry inconsistencies or titles listed before their official release year; the median is just **1 year**, so most content reaches Netflix very quickly.
- **Categorical cardinality:** 2 types, 14 ratings, **748 raw country combinations** (749 incl. "Unknown") (122 real countries after un-nesting), **514 genre combinations** (42 genres), **4,528 director strings** and **7,692 cast strings**, which is why un-nesting is essential.
- The most frequent single values: type = **Movie** (6,131), rating = **TV-MA** (3,207), country = **United States** (2,818 single-country titles), genre string = "Dramas, International Movies".

7. Non-graphical analysis: value counts & unique values

```
print('Unique values per column:');
display(df[raw.columns].nunique().to_frame('unique').T)

def vc(s, n=10):
    out = pd.DataFrame({'count': s.value_counts(), '%':
(s.value_counts(normalize=True) * 100).round(1)})
    return out.head(n)

print('Type'); display(vc(df['type']))
```

```
print('Rating'); display(vc(df['rating'], 14).T)
print('Audience group'); display(vc(df['audience']).T)
```

Unique values per column:

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
unique	8807	2	8806	4529	7693	749	1714	74	14	220	514	8775

Type

type	count	%
Movie	6131	69.6
TV Show	2676	30.4

Rating

rating	TV-MA	TV-14	TV-PG	R	PG-13	TV-Y7	TV-Y	PG	TV-G	NR	G	TV-Y7-FV	NC-17	UR
count	3207.0	2160.0	863.0	799.0	490.0	334.0	307.0	287.0	220.0	80.0	41.0	6.0	3.0	3.0
%	36.4	24.5	9.8	9.1	5.6	3.8	3.5	3.3	2.5	0.9	0.5	0.1	0.0	0.0

Audience group

audience	Adults	Teens	Older Kids	Little Kids	Unrated
count	4009.0	2650.0	1490.0	568.0	83.0
%	45.6	30.1	16.9	6.5	0.9

```
print('Release era'); display(vc(df['release_era']).sort_index().T)
print('TV seasons'); display(vc(df.loc[df.type == 'TV Show', 'duration'], 6).T)
print('Most common movie durations'); display(vc(df.loc[df.type == 'Movie',
'duration'], 6).T)
print('Titles added per year');
display(df['year_added'].value_counts().sort_index().to_frame('titles').T)
```

Release era

release_era	Before 1990	1990s	2000s	2010–15	2016–21
count	251.0	274.0	810.0	1816.0	5656.0
%	2.9	3.1	9.2	20.6	64.2

TV seasons

duration	1 Season	2 Seasons	3 Seasons	4 Seasons	5 Seasons	6 Seasons
count	1793.0	425.0	199.0	95.0	65.0	33.0
%	67.0	15.9	7.4	3.6	2.4	1.2

Most common movie durations

duration	90 min	94 min	97 min	93 min	91 min	95 min
count	152.0	146.0	146.0	146.0	144.0	137.0
%	2.5	2.4	2.4	2.4	2.3	2.2

Titles added per year

year_added	2008	2009	2010	2011	2012	2013	2014	2015	2016	2017	2018	2019	2020	2021
titles	2	2	1	13	3	11	24	82	429	1188	1649	2016	1879	1498

Observations

- **Type:** 69.6% movies (6,131) vs 30.4% TV shows (2,676), roughly **2.3 movies for every TV show**.

- **Rating:** TV-MA (36.4%) and TV-14 (24.5%) together make up **61%** of the catalogue. Grouped by audience, **Adults ≈ 46%** and **Teens ≈ 30%**, while **Little Kids is only 6.5%** and Older Kids 17%.
- **Release era:** the 2016–21 window alone holds **64%** of all titles (and 2010–15 another 21%), while pre-1990 titles are under 3%, confirming the focus on fresh content.
- **TV seasons:** "1 Season" is by far the most common value (1,793 shows, 67%); only ~9% of shows have 4+ seasons.
- **Movie durations:** the most common runtimes are 90–97 minutes.
- **Additions per year:** volumes explode from 82 (2015) to 429 (2016) to 1,188 (2017), peak at **2,016 in 2019**, then dip in 2020 (1,879). 2021 is a partial year (1,498 by late September).

8. Pre-processing: un-nesting cast, director, country & genre

A title like *"Dramas, International Movies"* belongs to **both** genres, and a film with 10 actors credits **all 10**. We split each list and give every value its own row (`explode`). Each un-nested table keeps `show_id` , so it can be joined back to any attribute.

```
def unnest(frame, col):
    'Split a comma-separated column into one row per value, keeping the title
attributes.'
    out = frame[['show_id', 'type', 'title', 'release_year', 'year_added', 'rating',
'audience', 'duration_num', col]].copy()
    out[col] = out[col].str.split(',')
    out = out.explode(col).reset_index(drop=True)
    out[col] = out[col].str.strip()
    return out[out[col].notna() & (out[col] != '')]

df_cast     = unnest(df, 'cast')
df_director = unnest(df, 'director')
df_country  = unnest(df, 'country')
df_genre    = unnest(df, 'listed_in').rename(columns={'listed_in': 'genre'})

known = lambda d, c: d[d[c] != 'Unknown']
summary = pd.DataFrame({
    'rows after un-nesting': [len(df_cast), len(df_director), len(df_country),
len(df_genre)],
    'unique values': [known(df_cast,'cast')['cast'].nunique(),
known(df_director,'director')['director'].nunique(),
                      known(df_country,'country')['country'].nunique(),
df_genre['genre'].nunique()]},
    index=['cast', 'director', 'country', 'genre'])
display(summary)
co = known(df_country, 'country').groupby('show_id').size()
print(f'Titles produced by 2+ countries (co-productions): {(co > 1).sum():,}')
print(f'Average genres per title: {len(df_genre) / len(df):.2f}  |  Average cast per
title (known): {len(known(df_cast,"cast")) / (df.cast != "Unknown").sum():.1f}')
```

	rows after un-nesting	unique values
cast	64951	36439
director	9612	4993
country	10843	122
genre	19323	42

```
Titles produced by 2+ countries (co-productions): 1,315
Average genres per title: 2.19  |  Average cast per title (known): 8.0
```

```
for name, d, c in [('Top countries', known(df_country,'country'), 'country'), ('Top genres', df_genre, 'genre'),
                   ('Top actors', known(df_cast,'cast'), 'cast'), ('Top directors', known(df_director,'director'), 'director')]:
    print(name); display(d[c].value_counts().head(8).to_frame('titles').T)
```

```
Top countries
```

country	United States	India	United Kingdom	Canada	France	Japan	Spain	South Korea
titles	3690	1046	806	445	393	318	232	231

```
Top genres
```

genre	International Movies	Dramas	Comedies	International TV Shows	Documentaries	Action & Adventure	TV Dramas	Independent Movies
titles	2752	2427	1674	1351	869	859	763	756

```
Top actors
```

cast	Anupam Kher	Shah Rukh Khan	Julie Tejwani	Naseeruddin Shah	Takahiro Sakurai	Rupa Bhimani	Akshay Kumar	Om Puri
titles	43	35	33	32	32	31	30	30

```
Top directors
```

director	Rajiv Chilaka	Jan Suter	Raúl Campos	Suhas Kadav	Marcus Raboy	Jay Karas	Cathy Garcia-Molina	Youssef Chahine
titles	22	21	19	16	16	15	13	12

```
# Save the cleaned and un-nested tables so others can reuse them without re-running the cleaning
out = Path('../data/processed'); out.mkdir(parents=True, exist_ok=True)
df.to_csv(out / 'netflix_cleaned.csv', index=False)
for name, d in [('cast', df_cast), ('director', df_director), ('country', df_country), ('genre', df_genre)]:
    d.to_csv(out / f'netflix_{name}_unnested.csv', index=False)
print(sorted(p.name for p in out.iterdir()))
```

```
['netflix_cast_unnested.csv', 'netflix_cleaned.csv', 'netflix_country_unnested.csv', 'netflix_director_unnested.csv', 'netflix_genre_unnested.csv']
```

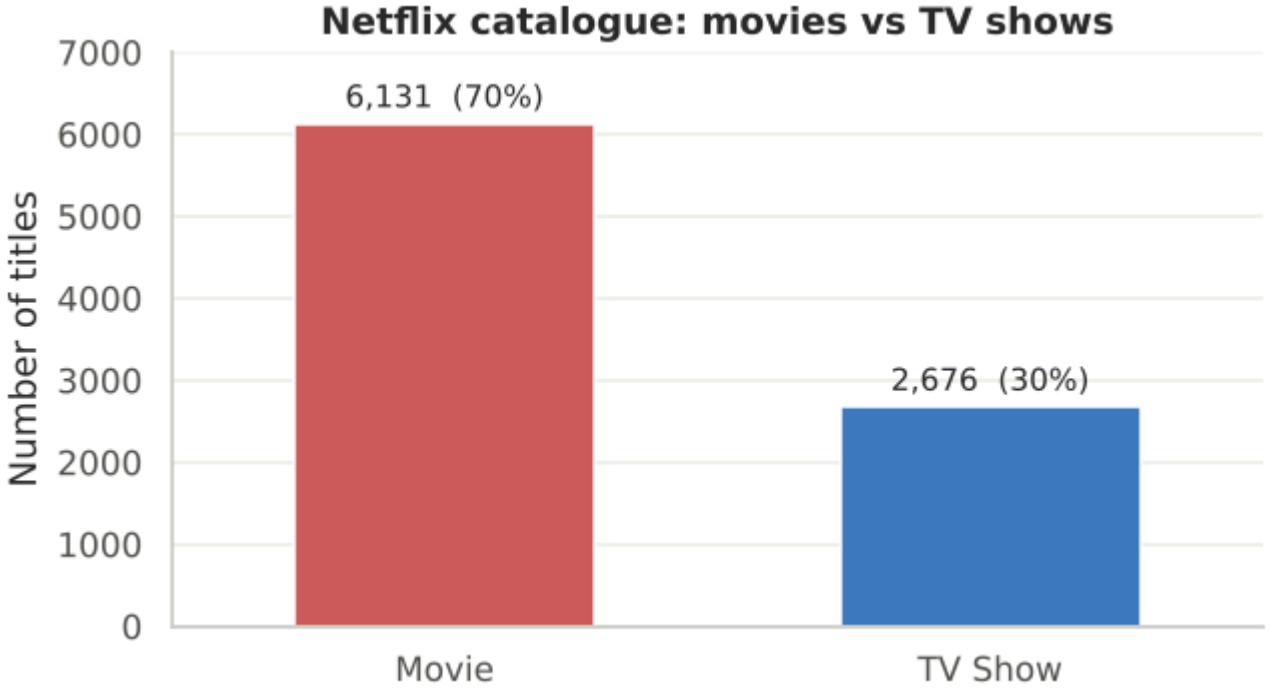
Observations

- After un-nesting there are **122 countries, 42 genres, ~36,400 actors and ~5,000 directors**.
- **1,315 titles (15%) are co-productions** between two or more countries, a sign that Netflix uses partnerships to enter markets.
- The average title is tagged with **~2.2 genres** and lists **~8 cast members**, so un-nested counts add up to more than the number of titles.
- The United States leads every country ranking, but **India is a clear #2** and Indian actors (Anupam Kher, Shah Rukh Khan, Naseeruddin Shah) dominate the top-actor list.

9. Univariate visual analysis

9.1 Movies vs TV shows (countplot)

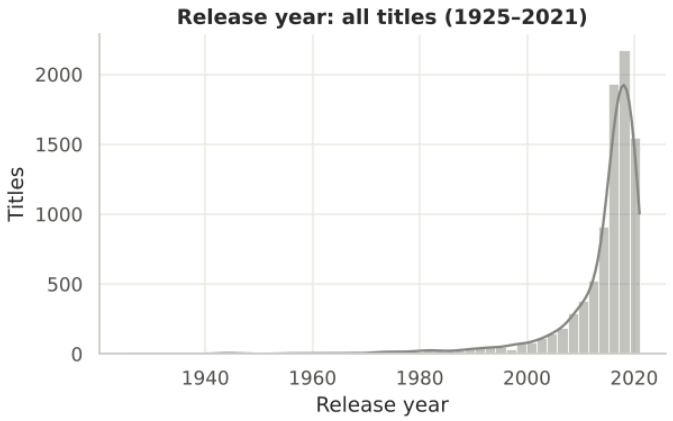
```
fig, ax = plt.subplots(figsize=(6, 3.4))
order = ['Movie', 'TV Show']
sns.countplot(data=df, x='type', order=order, palette=PAL, width=0.55, ax=ax)
for p, t in zip(ax.patches, order):
    n = (df.type == t).sum(); ax.annotate(f'{n:,}  ({n/len(df):.0%})',
(p.get_x()+p.get_width()/2, p.get_height()), ha='center', va='bottom', fontsize=10,
xytext=(0, 3), textcoords='offset points')
ax.set(title='Netflix catalogue: movies vs TV shows', xlabel='', ylabel='Number of
titles', ylim=(0, 7000))
sns.despine(); save('02_type_count')
```

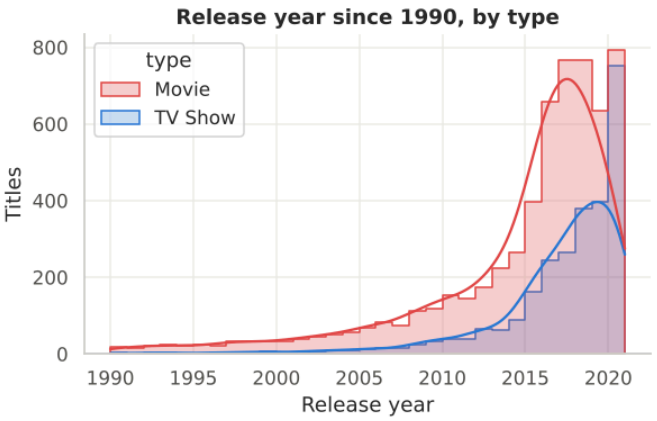


Insight: Movies make up **70%** of the catalogue (6,131) vs **30%** for TV shows (2,676). Netflix's library is built on movies, yet TV shows are what keep subscribers returning month after month (multi-episode viewing), so this mix is worth watching over time (see 10.1).

9.2 Release year distribution (distplot / histogram with KDE)

```
fig, axes = plt.subplots(1, 2, figsize=(11, 3.6))
sns.histplot(df['release_year'], bins=50, kde=True, color=MUTED, ax=axes[0])
axes[0].set(title='Release year: all titles (1925–2021)', xlabel='Release year',
ylabel='Titles')
sns.histplot(data=df[df.release_year >= 1990], x='release_year', hue='type',
palette=PAL, binwidth=1,
             kde=True, element='step', alpha=0.25, ax=axes[1])
axes[1].set(title='Release year since 1990, by type', xlabel='Release year',
ylabel='Titles')
sns.despine(); save('03_release_year_dist')
```



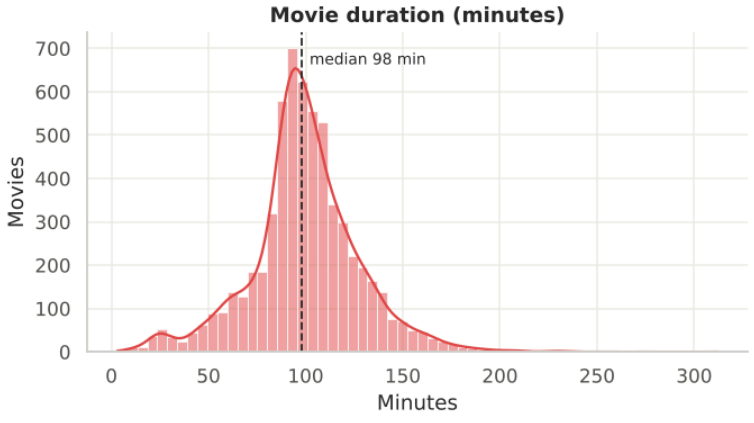


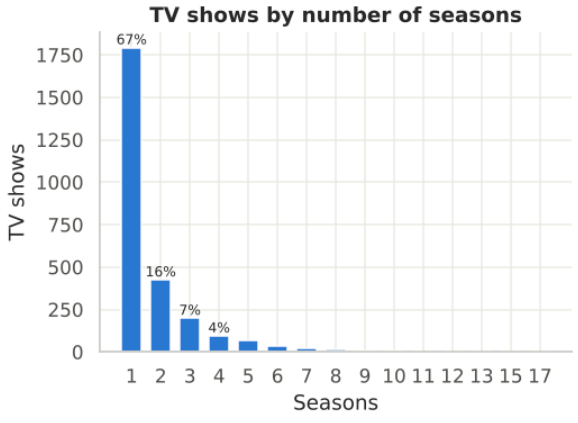
Insights

- The distribution is **strongly left-skewed**: a long thin tail back to 1925 and a sharp peak in **2017–2018**. Half of all titles were released in 2017 or later.
- Movies peak in **2017–2018 (767 each)**; TV shows keep rising until **2020 (436)**. In 2021 there are already **more TV shows (315) than movies (277)** by release year.
- The drop in 2020–21 is partly COVID-related production delays and partly because the data stops in September 2021.

9.3 Movie duration (distplot + boxplot) and TV seasons (countplot)

```
movies, shows = df[df.type == 'Movie'], df[df.type == 'TV Show']
fig, axes = plt.subplots(1, 2, figsize=(11, 3.6), gridspec_kw={'width_ratios': [1.4,
1]})
sns.histplot(movies['duration_num'], bins=60, kde=True, color=MOVIE, ax=axes[0])
med = movies['duration_num'].median()
axes[0].axvline(med, color=INK, ls='--', lw=1.2); axes[0].text(med + 4,
axes[0].get_ylim()[1]*0.9, f'median {med:.0f} min', fontsize=9)
axes[0].set(title='Movie duration (minutes)', xlabel='Minutes', ylabel='Movies')
s = shows['duration_num'].value_counts().sort_index()
axes[1].bar(s.index.astype(int).astype(str), s.values, color=TV, width=0.7)
axes[1].set(title='TV shows by number of seasons', xlabel='Seasons', ylabel='TV
shows')
for i, v in enumerate(s.values[:4]): axes[1].text(i, v + 20, f'{v/len(shows):.0%}',
ha='center', fontsize=8)
sns.despine(); save('04_duration_dist')
```



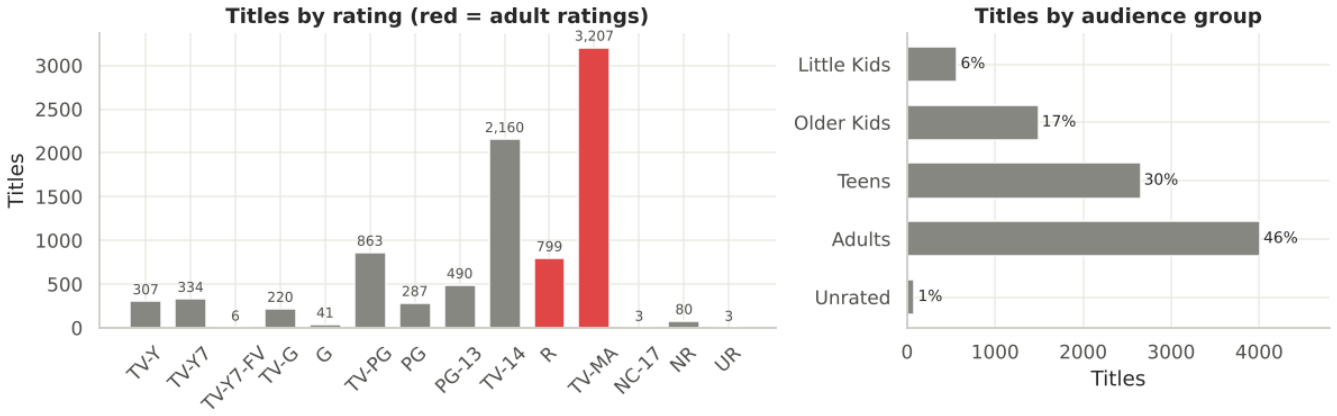


Insights

- Movie runtimes are **bell-shaped around 90–110 minutes** (median 98, mean 99.6). The middle 50% run **87–114 min**. Short films / stand-up (<60 min) and epics (>150 min, mostly Indian films) form the tails.
- TV is the opposite shape: **67% of shows have 1 season**, 16% have 2 and the share keeps halving. Few shows earn renewals, either because Netflix cancels early or because many are limited series / licensed single seasons.

9.4 Ratings & audience groups (countplot)

```
fig, axes = plt.subplots(1, 2, figsize=(11, 3.6), gridspec_kw={'width_ratios': [1.6,
1]})
r = df['rating'].value_counts().reindex(rating_order)
axes[0].bar(r.index, r.values, color=[MOVIE if x in ('TV-MA','R','NC-17') else MUTED
for x in r.index], width=0.7)
axes[0].set(title='Titles by rating (red = adult ratings)', ylabel='Titles');
axes[0].tick_params(axis='x', rotation=45)
label_bars(axes[0])
a = df['audience'].value_counts().reindex(['Little Kids', 'Older Kids', 'Teens',
'Adults', 'Unrated'])
axes[1].barh(a.index[::-1], a.values[::-1], color=MUTED, height=0.6)
for i, v in enumerate(a.values[::-1]): axes[1].text(v + 40, i, f'{v/a.sum():.0%}',
va='center', fontsize=9)
axes[1].set(title='Titles by audience group', xlabel='Titles', xlim=(0, 4800))
sns.despine(); save('05_ratings')
```



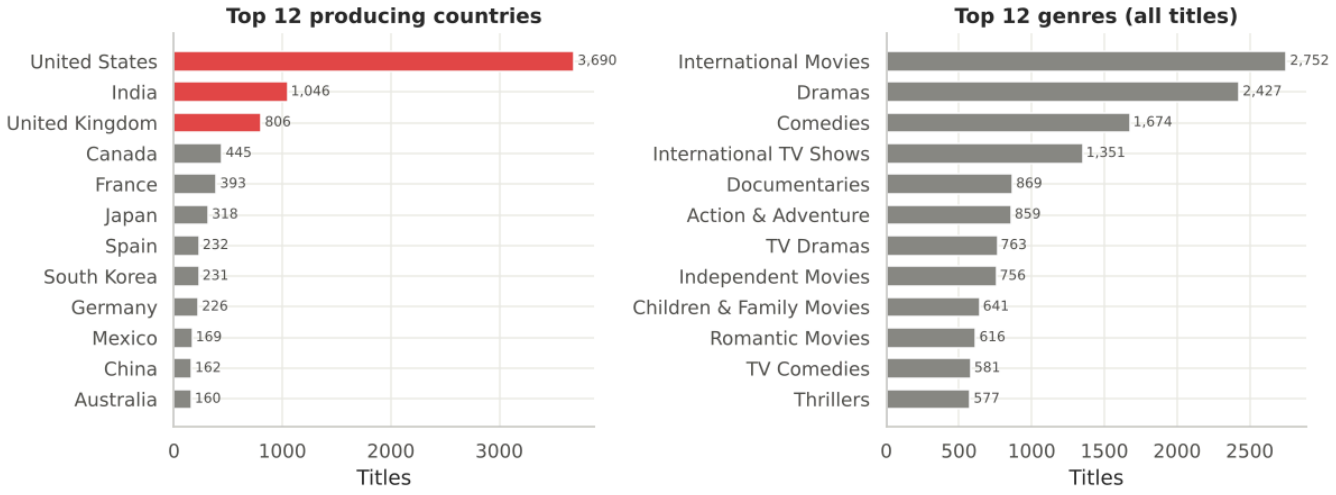
Insights

- **TV-MA (3,207) is the single largest rating**, followed by TV-14 (2,160). Adult-rated titles (TV-MA, R, NC-17) make up **~46%** of the catalogue.
- **Little Kids content is only 6.5%** and older kids 17%. Families are a large subscriber segment, so this is a potential **gap**.
- Movie-style ratings (G, PG, PG-13, R) are much rarer than TV ratings, because most Netflix content is rated under the TV system, including movies.

9.5 Top countries & genres (countplot of un-nested data)

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
c = known(df_country, 'country')['country'].value_counts().head(12)
axes[0].barh(c.index[::-1], c.values[::-1], color=[MOVIE if i < 3 else MUTED for i
in range(len(c))][::-1], height=0.65)
```

```
axes[0].set(title='Top 12 producing countries', xlabel='Titles');
label_bars(axes[0])
g = df_genre['genre'].value_counts().head(12)
axes[1].barh(g.index[::-1], g.values[::-1], color=MUTED, height=0.65)
axes[1].set(title='Top 12 genres (all titles)', xlabel='Titles');
label_bars(axes[1])
sns.despine(); save('06_top_countries_genres')
```

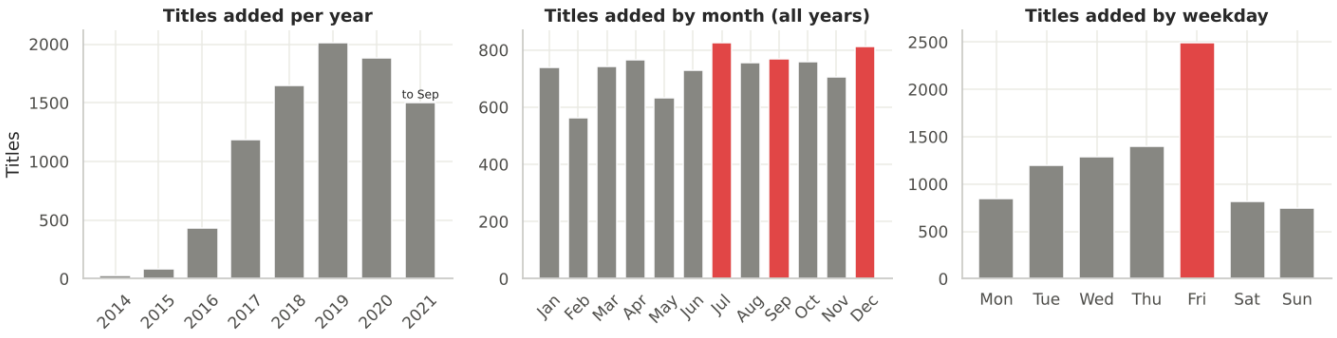


Insights

- **The US (3,690) is ~3.5× India (1,046)**, and the top 3 (US, India, UK) account for more than half of all country credits. Canada, France, Japan, Spain, South Korea and Germany form a second tier with 200–450 titles each.
- **"International Movies" (2,752) and "Dramas" (2,427)** are the biggest genres, followed by Comedies (1,674). Netflix's identity is **drama + non-English content**, while documentaries and action are mid-sized.

9.6 When are titles added? (countplots by year, month, weekday)

```
fig, axes = plt.subplots(1, 3, figsize=(13, 3.4))
y = df['year_added'].value_counts().sort_index().loc[2014:]
axes[0].bar(y.index.astype(str), y.values, color=MUTED, width=0.7);
axes[0].set(title='Titles added per year', ylabel='Titles')
axes[0].tick_params(axis='x', rotation=45); axes[0].text(len(y)-1, y.iloc[-1] + 40,
'to Sep', ha='center', fontsize=8)
m = df['month_added'].value_counts().sort_index()
axes[1].bar(pd.to_datetime(m.index.astype(int), format='%m').strftime('%b'),
m.values, color=[MOVIE if v >= m.nlargest(3).min() else MUTED for v in m.values],
width=0.7)
axes[1].set(title='Titles added by month (all years)');
axes[1].tick_params(axis='x', rotation=45)
w = df['weekday_added'].value_counts().sort_index()
axes[2].bar(w.index.str[:3], w.values, color=[MOVIE if d == 'Friday' else MUTED for
d in w.index], width=0.7)
axes[2].set(title='Titles added by weekday')
sns.despine(); save('07_when_added')
```



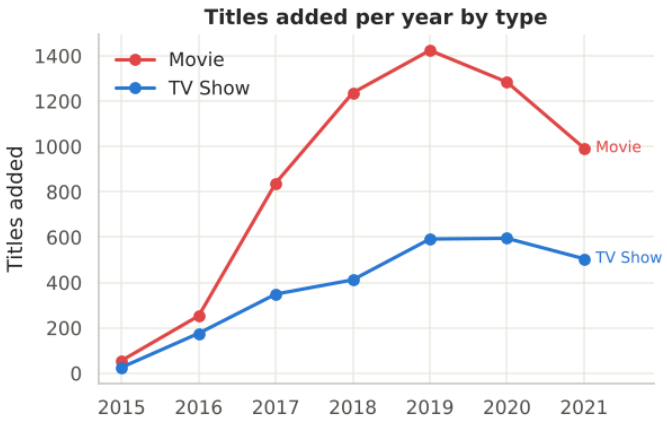
Insights

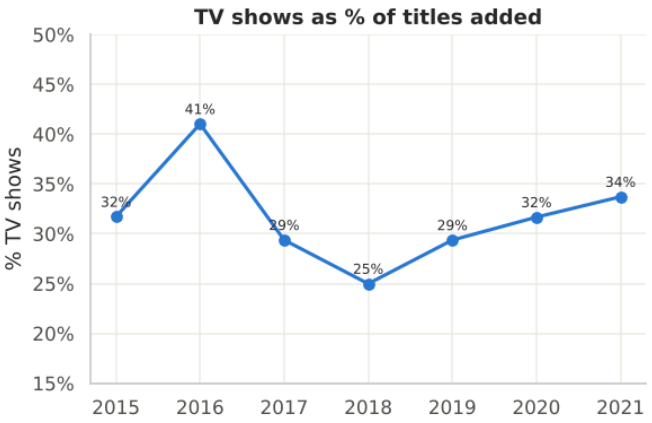
- **Additions grew ~25× from 2015 to 2019**, then dipped 7% in 2020 (COVID production shutdowns). The 2021 bar covers only ~8.8 months; at its run-rate (~170/month) 2021 would land around 2,000, roughly flat versus 2019–20. Growth has plateaued.
- **By month, additions are fairly even**, but **July (827), December (813) and September (770)** are the strongest; **February (563) is the weakest** month.
- **Friday is the launch day by far (~28% of all additions)**, almost double any other weekday, which lines up with weekend binge-watching. The **1st of the month** is also a major drop date (2,212 titles), because licensing deals usually start on the 1st.

10. Bivariate & multivariate visual analysis

10.1 Is Netflix focusing more on TV shows in recent years?

```
t = pd.crosstab(df['year_added'], df['type']).loc[2015:]
share = (t['TV Show'] / t.sum(axis=1) * 100)
fig, axes = plt.subplots(1, 2, figsize=(11, 3.6))
for col in ['Movie', 'TV Show']:
    axes[0].plot(t.index.astype(int), t[col], marker='o', lw=2, ms=6,
color=PAL[col], label=col)
    axes[0].text(2021.15, t[col].iloc[-1], col, color=PAL[col], va='center',
fontsize=9)
axes[0].set(title='Titles added per year by type', ylabel='Titles added', xlim=
(2014.7, 2021.9)); axes[0].legend(loc='upper left', frameon=False)
axes[1].plot(share.index.astype(int), share.values, marker='o', lw=2, ms=6,
color=TV)
for x, v in share.items(): axes[1].text(x, v + 1, f'{v:.0f}%', ha='center',
fontsize=8)
axes[1].set(title='TV shows as % of titles added', ylabel='% TV shows', ylim=(15,
50))
axes[1].yaxis.set_major_formatter(mtick.PercentFormatter(decimals=0))
sns.despine(); save('08_tv_vs_movie_trend')
```



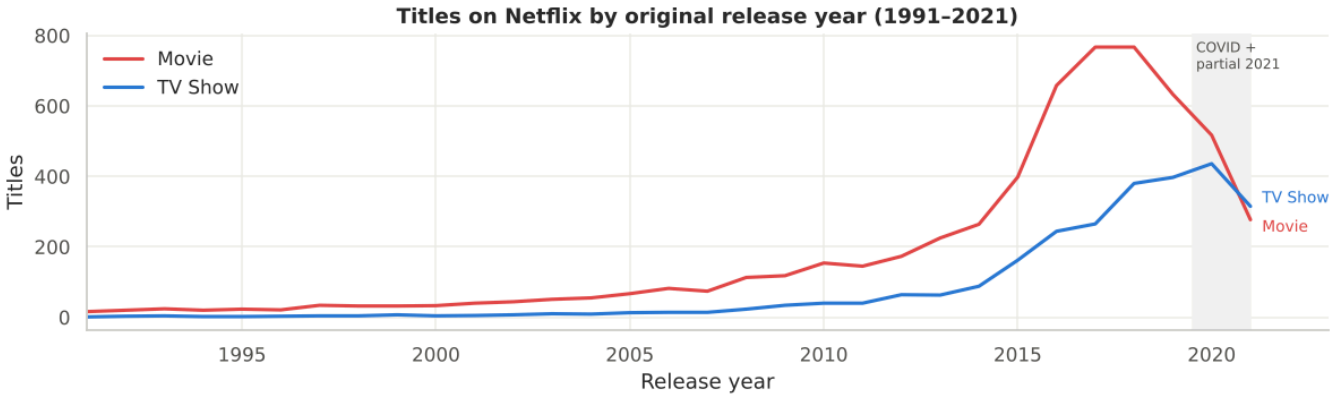


Insights

- **Yes, the shift to TV is real.** After a 2016 spike (41%, the global-launch catalogue), the TV share fell to **25% in 2018** and has climbed every year since: **29% → 32% → 34% (2021)**.
- Movie additions **fell 10% from 2019 to 2020 (1,424 → 1,284)** while TV additions held flat (592 → 595). Netflix protected its TV pipeline even as overall additions slowed.

10.2 How has the number of releases changed over the last 30 years? (release year × type)

```
r = pd.crosstab(df['release_year'], df['type']).loc[1991:2021]
fig, ax = plt.subplots(figsize=(11, 3.4))
for col in ['Movie', 'TV Show']:
    ax.plot(r.index, r[col], lw=2, color=PAL[col], label=col)
    ax.text(2021.3, r[col].iloc[-1], col, color=PAL[col], va='bottom' if col == 'TV
Show' else 'top', fontsize=9)
ax.axvspan(2019.5, 2021, color='#f3f3f1'); ax.text(2019.6, r.values.max()*0.92,
'COVID +\npartial 2021', fontsize=8, color='#52514e')
ax.set(title='Titles on Netflix by original release year (1991–2021)',
xlabel='Release year', ylabel='Titles', xlim=(1991, 2023))
ax.legend(frameon=False, loc='upper left'); sns.despine();
save('09_release_trend_30y')
display(r.loc[[1991, 2001, 2011, 2016, 2018, 2020, 2021]].T)
```



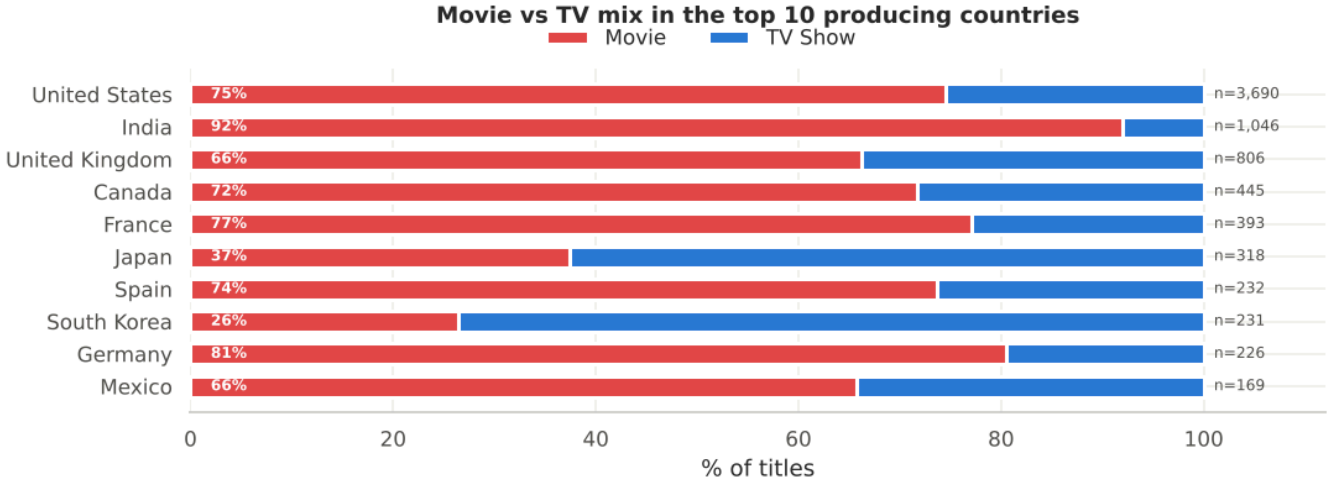
release_year	1991	2001	2011	2016	2018	2020	2021
type							
Movie	16	40	145	658	767	517	277
TV Show	1	5	40	244	380	436	315

Insights

- Titles from the 1990s are scarce (20–35 movies per year). Growth accelerates after **2008** and explodes after **2014**: movies released in 2018 (767) outnumber those from 2008 (113) almost 7 to 1.
- **Movies peaked with 2017–18 releases and have declined since; TV shows grew every year up to 2020** and overtook movies for 2021 releases. Netflix's *newest* content is increasingly episodic.

10.3 What content is available in different countries? (country × type, country × genre)

```
top10 = known(df_country, 'country')['country'].value_counts().head(10).index
ct = pd.crosstab(df_country['country'], df_country['type']).loc[top10]
ctp = ct.div(ct.sum(axis=1), axis=0) * 100
fig, ax = plt.subplots(figsize=(10, 3.8))
ax.barh(ctp.index[::-1], ctp['Movie'][::-1], color=MOVIE, label='Movie',
height=0.65, edgecolor='white', linewidth=2)
ax.barh(ctp.index[::-1], ctp['TV Show'][::-1], left=ctp['Movie'][::-1], color=TV,
label='TV Show', height=0.65, edgecolor='white', linewidth=2)
for i, (mv_, n) in enumerate(zip(ctp['Movie'][::-1], ct.sum(axis=1)[::-1])):
    ax.text(2, i, f'{mv_:.0f}%', va='center', color='white', fontsize=8,
fontweight='bold')
    ax.text(101, i, f'n={n:,}', va='center', fontsize=8, color='#52514e')
ax.set(title='Movie vs TV mix in the top 10 producing countries', xlabel='% of
titles', xlim=(0, 112))
ax.set_title(ax.get_title(), pad=24); ax.legend(frameon=False, ncol=2, loc='lower
center', bbox_to_anchor=(0.45, 1.0)); sns.despine(left=True);
save('10_country_type_mix')
```

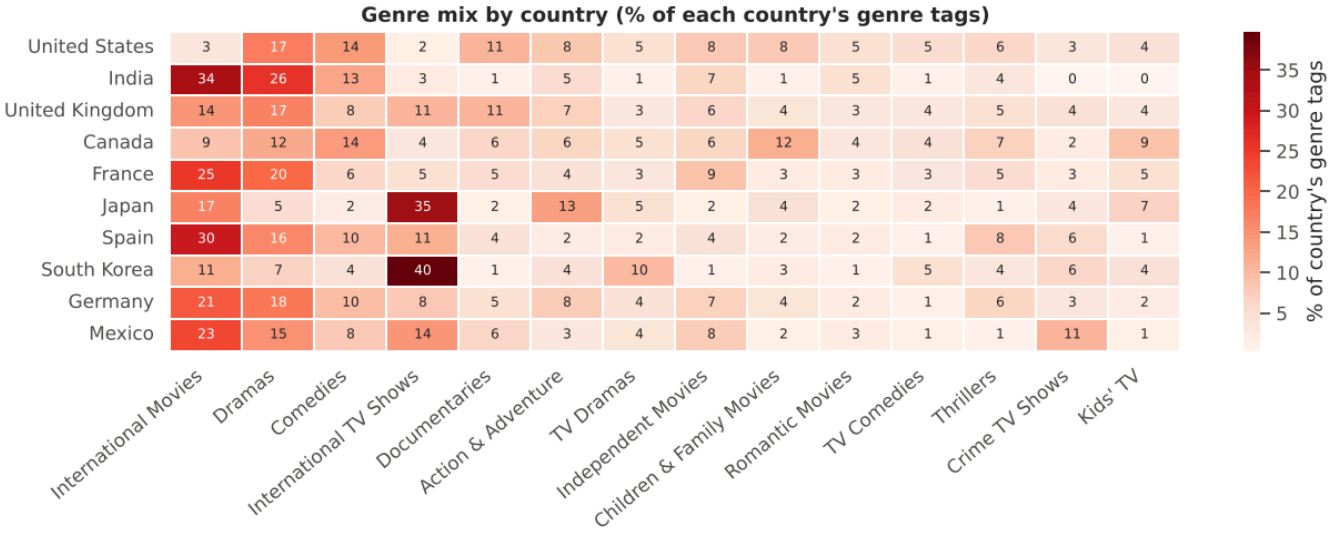


Insights

- **India is 92% movies**, the most movie-centric major market (Bollywood). Egypt, Hong Kong and Nigeria (not shown) are similar.
- **Japan (63% TV) and South Korea (74% TV)** are the only top-10 markets where TV dominates, driven by **anime and K-dramas**.
- The **UK (34% TV)** is more TV-oriented than the US (25%), reflecting its strength in British series.

```
cg = known(df_country, 'country')[['show_id', 'country']].merge(df_genre[['show_id',
'genre']], on='show_id')
```

```
top_genres = df_genre['genre'].value_counts().head(14).index
heat = pd.crosstab(cg['country'], cg['genre']).loc[top10, top_genres]
heat_pct = heat.div(heat.sum(axis=1), axis=0) * 100
fig, ax = plt.subplots(figsize=(12, 4.6))
sns.heatmap(heat_pct, annot=True, fmt='.0f', cmap='Reds', linewidths=1,
linecolor='white', cbar_kws={'label': '% of country\'s genre tags'}, ax=ax,
annot_kws={'size': 8})
ax.set(title='Genre mix by country (% of each country\'s genre tags)', xlabel='',
ylabel='')
plt.xticks(rotation=40, ha='right'); save('11_country_genre_heatmap')
```



Insights

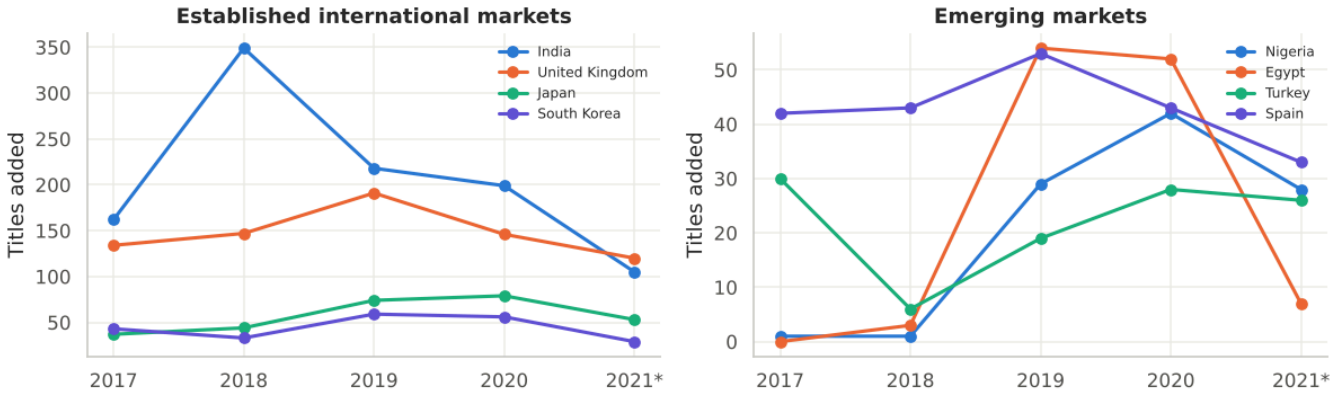
- **US:** the most balanced mix: Dramas (17%), Comedies (14%), **Documentaries (11%)** and **Children & Family (8%)**, with almost no "International" tag (it is the home market). A "something for everyone" market.
- **India: International Movies (34%) + Dramas (26%) + Comedies (13%)** make up 73% of tags; documentaries, kids' and TV genres are ~1% each, so there is clear room to diversify.
- **UK:** balanced between movies and TV, with a strong **Documentaries (11%)** share, equal to the US.
- **Japan: International TV (35%)** and **Action & Adventure (13%)**, the highest action share of any market, driven by anime (anime genres sit just outside this top-14 view).
- **South Korea: International TV (40%)** and **TV Dramas (10%)**: the K-drama factory.
- **Canada:** the highest **Children & Family (12%) and Kids' TV (9%)** shares, a known animation hub.
- **Mexico & Spain: Crime TV (Mexico 11%, Spain 6%) and Thrillers (Spain 8%)** are well above average, so crime/thriller is a proven Spanish-language strength.

10.4 Which countries are growing fastest on Netflix?

```
cy = known(df_country, 'country').dropna(subset=['year_added'])
cy = pd.crosstab(cy['country'], cy['year_added'].astype(int))
growth = cy[[2017, 2018, 2019, 2020, 2021]]
watch = ['United States', 'India', 'United Kingdom', 'Japan', 'South Korea',
'Spain', 'Nigeria', 'Egypt', 'Turkey', 'Mexico']
display(growth.loc[watch])
fig, axes = plt.subplots(1, 2, figsize=(11, 3.4), sharex=True)
for ax, group, title in [(axes[0], ['India', 'United Kingdom', 'Japan', 'South
```

```
Korea'], 'Established international markets'),
                          (axes[1], ['Nigeria', 'Egypt', 'Turkey', 'Spain'],
'Emerging markets')]:
    for i, c_ in enumerate(group):
        col = ['#2a78d6', '#eb6834', '#1baf7a', '#6250d6'][i]
        ax.plot(growth.columns, growth.loc[c_], marker='o', lw=2, ms=6, color=col,
label=c_)
    ax.set(title=title, ylabel='Titles added'); ax.legend(frameon=False, fontsize=8)
axes[0].set_xticks([2017, 2018, 2019, 2020, 2021]);
axes[0].set_xticklabels(['2017','2018','2019','2020','2021*'])
sns.despine(); save('12_country_growth')
```

year_added	2017	2018	2019	2020	2021
country					
United States	462	600	856	828	627
India	162	349	218	199	105
United Kingdom	134	147	191	146	120
Japan	37	44	74	79	53
South Korea	43	33	59	56	29
Spain	42	43	53	43	33
Nigeria	1	1	29	42	28
Egypt	0	3	54	52	7
Turkey	30	6	19	28	26
Mexico	36	31	39	30	21



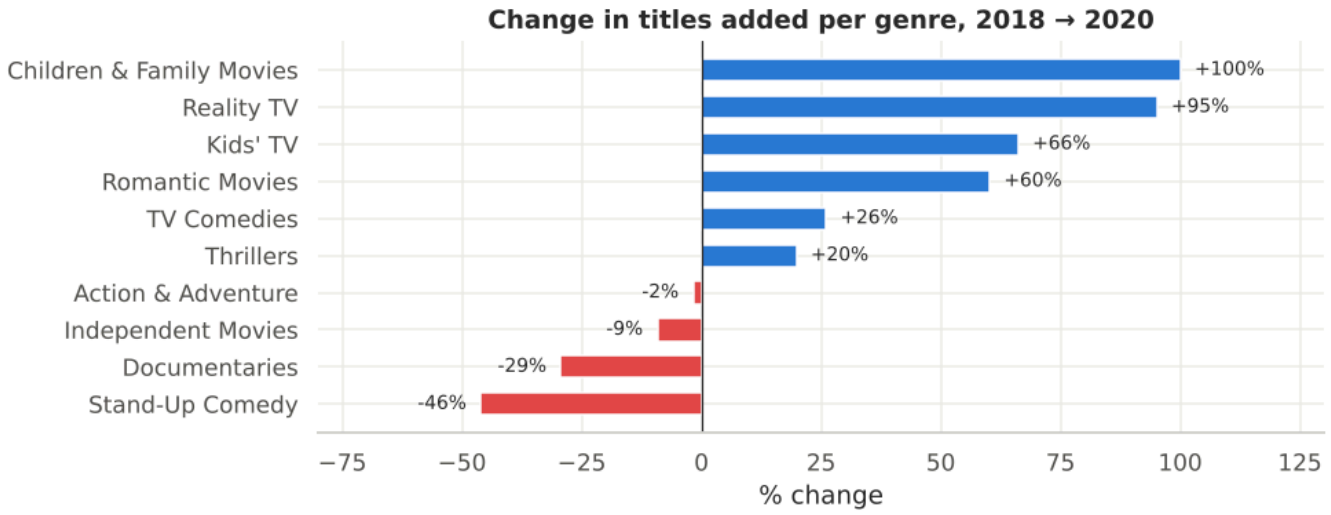
Insights (*2021 is January–September only*)

- **India peaked in 2018 (349 titles) and fell by ~43% in 2020 (199).** After an early licensing push, Indian additions slowed; the opportunity now is quality originals rather than volume.
- **Japan (44 → 79) and South Korea (33 → 56) grew 70–80% from 2018 to 2020**, riding global demand for anime and K-dramas.
- **Nigeria (1 → 42) and Egypt (3 → 54)** went from almost nothing in 2018 to meaningful volumes by 2020: **new growth frontiers** in Africa and the Middle East. Nigeria kept going in 2021 (28 by September), while Egypt's 2019–20 jump was a one-off batch (only 7 in 2021). Spain is steady (33–53 titles a year) and Turkey recovered to 26–28 a year after a dip in 2018.

10.5 Which genres are growing? (genre × year added)

```
gy = df_genre.dropna(subset=['year_added']); gy = pd.crosstab(gy['genre'],
gy['year_added'].astype(int))
sel = ['Reality TV', "Kids' TV", 'Children & Family Movies', 'Romantic Movies', 'TV
```

```
Comedies', 'Action & Adventure', 'Thrillers',
       'Documentaries', 'Stand-Up Comedy', 'Independent Movies']
idx = (gy.loc[sel, 2020] / gy.loc[sel, 2018] * 100 - 100).sort_values()
fig, ax = plt.subplots(figsize=(9, 3.6))
ax.barh(idx.index, idx.values, color=[TV if v > 0 else MOVIE for v in idx.values],
height=0.6)
for i, v in enumerate(idx.values): ax.text(v + (3 if v > 0 else -3), i,
f'{v:+.0f}%', va='center', ha='left' if v > 0 else 'right', fontsize=9)
ax.axvline(0, color=INK, lw=0.8); ax.set(title='Change in titles added per genre,
2018 → 2020', xlabel='% change', xlim=(-80, 130))
sns.despine(left=True); save('13_genre_growth')
```

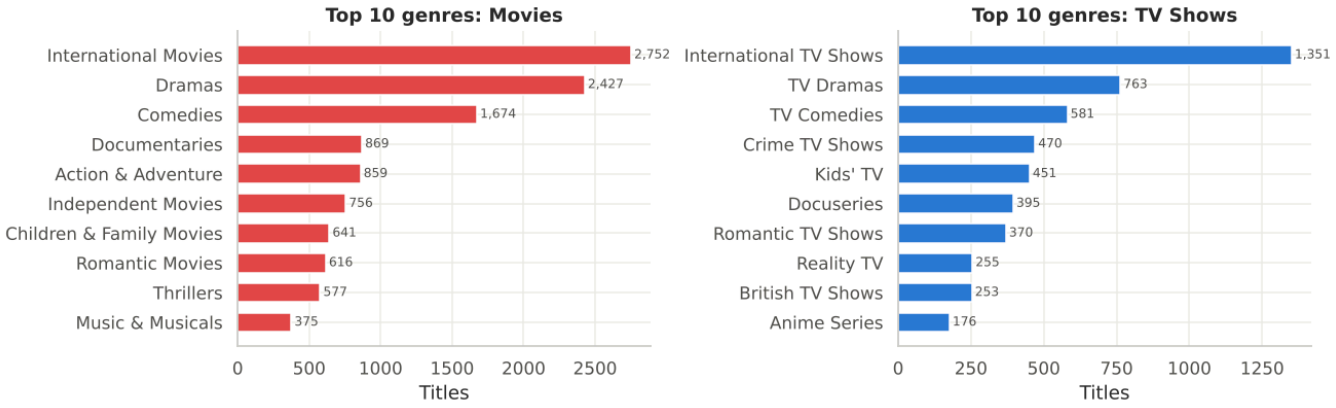


Insights

- **Fastest-growing genres: Reality TV (+95%)**, **Children & Family Movies (+100%)**, **Kids' TV (+66%)**, **Romantic Movies (+60%)** and TV Comedies (+26%). Netflix is clearly building **family, light-entertainment and unscripted** content.
- **Shrinking genres: Stand-Up Comedy (−46%)**, **Documentaries (−29%)** and Independent Movies (−9%).
- Action & Adventure dipped in 2020 but by September 2021 had already passed its 2020 total (196 vs 170), a signal of renewed **blockbuster** investment.

10.6 Genres for movies vs TV shows

```
fig, axes = plt.subplots(1, 2, figsize=(12, 3.8))
for ax, t_, col in [(axes[0], 'Movie', MOVIE), (axes[1], 'TV Show', TV)]:
    s = df_genre.loc[df_genre.type == t_, 'genre'].value_counts().head(10)
    ax.barh(s.index[::-1], s.values[::-1], color=col, height=0.65); label_bars(ax)
    ax.set(title=f'Top 10 genres: {t_}s', xlabel='Titles')
sns.despine(); save('14_genres_by_type')
```

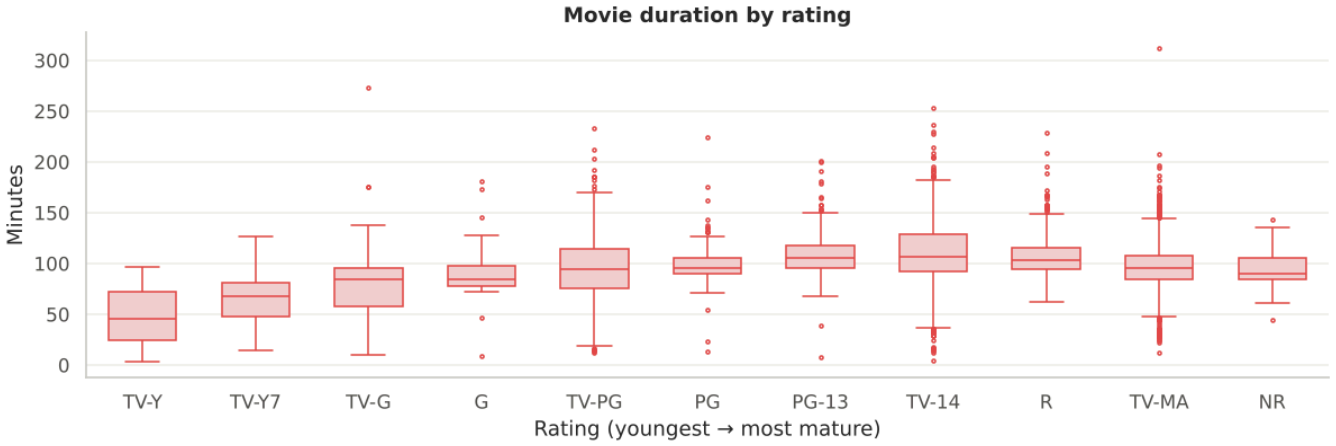


Insights

- **Movies:** International (2,752), Dramas (2,427) and Comedies (1,674) lead, then Documentaries, Action, Independent and Children & Family.
- **TV:** International TV (1,351), TV Dramas (763), TV Comedies (581), then **Crime TV (470)** and **Kids' TV (451)**. Crime is notably stronger on TV than thrillers are among movies, a natural fit for multi-episode storytelling.
- Both formats share a core of **international + drama + comedy**; TV adds Crime, Kids, Docuseries, Reality and **Anime (176)** as distinct niches.

10.7 Boxplots: movie duration by rating & by genre

```
mr = movies.dropna(subset=['rating']).copy(); mr['rating'] =
mr['rating'].cat.remove_unused_categories()
keep = mr['rating'].value_counts()[lambda s: s >= 30].index
fig, ax = plt.subplots(figsize=(11, 3.8))
sns.boxplot(data=mr[mr.rating.isin(keep)], x='rating', y='duration_num', order=[r
for r in rating_order if r in keep],
            color='#f6c9c8', linecolor=MOVIE, fliersize=2, width=0.6, ax=ax)
ax.set(title='Movie duration by rating', xlabel='Rating (youngest → most mature)',
ylabel='Minutes'); sns.despine(); save('15_box_duration_rating')
display(mr.groupby('rating', observed=True)['duration_num'].median().loc[[r for r in
rating_order if r in keep]].to_frame('median minutes').T)
```

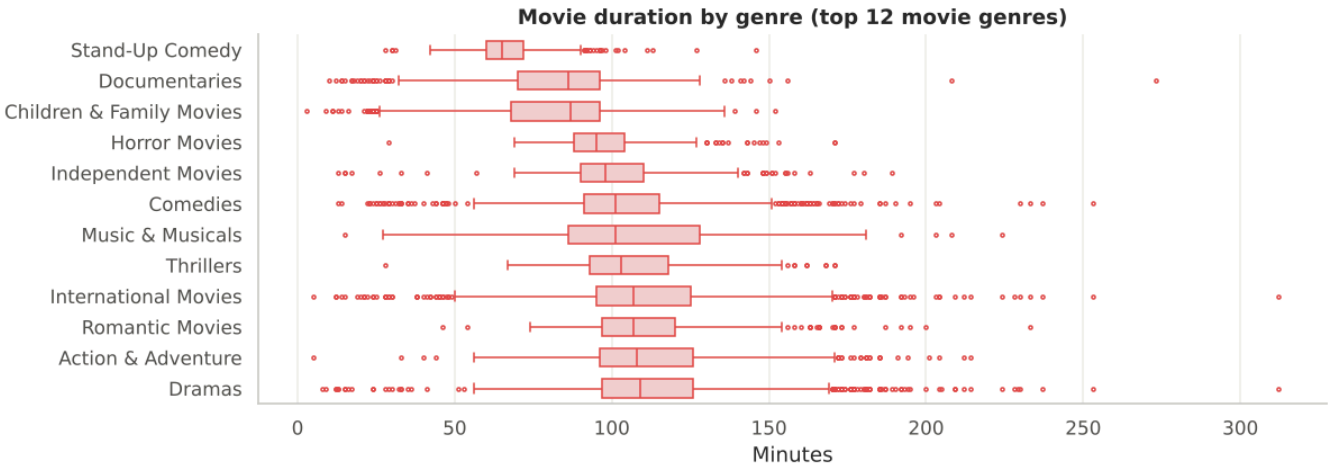


rating	TV-Y	TV-Y7	TV-G	G	TV-PG	PG	PG-13	TV-14	R	TV-MA	NR
median minutes	46.0	68.0	84.0	84.0	94.0	95.0	106.0	107.0	103.0	96.0	90.0

Insights

- **Duration rises with audience age.** Kids' movies (TV-Y median **46 min**, TV-Y7 68 min) are far shorter than teen/adult films (PG-13 **106**, TV-14 **107**, R **103 min**).
- TV-14 shows the **widest spread and most long outliers** (many 150–200+ min Indian films carry TV-14).
- Adult-rated content clusters tightly around 90–110 minutes, a reliable production target.

```
gm = df_genre[(df_genre.type == 'Movie')]
gsel = gm['genre'].value_counts().head(12).index
order_g = gm[gm.genre.isin(gsel)].groupby('genre')
['duration_num'].median().sort_values().index
fig, ax = plt.subplots(figsize=(11, 4))
sns.boxplot(data=gm[gm.genre.isin(gsel)], y='genre', x='duration_num',
order=order_g, color='#f6c9c8', linecolor=MOVIE, fliersize=2, width=0.6, ax=ax)
ax.set(title='Movie duration by genre (top 12 movie genres)', xlabel='Minutes',
ylabel=''); sns.despine(); save('16_box_duration_genre')
```

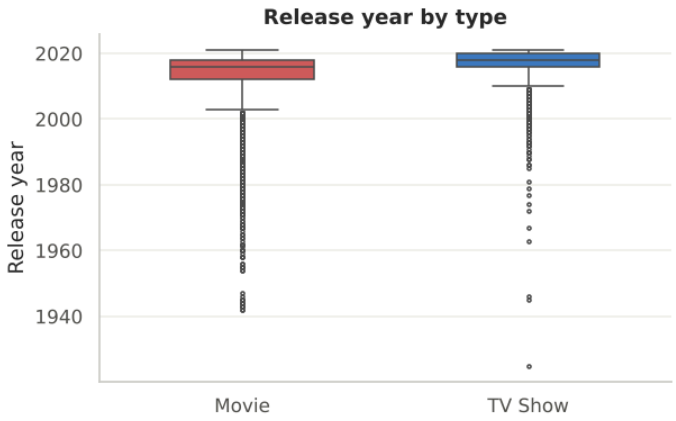


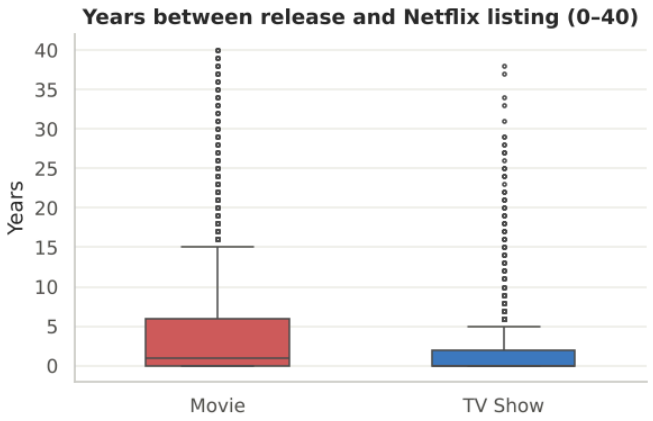
Insights

- **Stand-Up Comedy (median ~65 min)**, **Documentaries (~86)** and **Children & Family (~87)** are the shortest formats, which makes them cheap, fast and good "filler" between big releases.
- **Dramas (~109), Action & Adventure (~108), International (~107) and Romantic Movies (~107)** run longest; international films carry the largest high-end outliers (3-hour Indian epics).

10.8 Boxplots: release year and 'years to Netflix' by type

```
fig, axes = plt.subplots(1, 2, figsize=(11, 3.6))
sns.boxplot(data=df, x='type', y='release_year', palette=PAL, order=['Movie','TV
Show'], fliersize=2, width=0.5, ax=axes[0])
axes[0].set(title='Release year by type', xlabel='', ylabel='Release year')
d2 = df[df.years_to_netflix.between(0, 40)]
sns.boxplot(data=d2, x='type', y='years_to_netflix', palette=PAL, order=['Movie','TV
Show'], fliersize=2, width=0.5, ax=axes[1])
axes[1].set(title='Years between release and Netflix listing (0–40)', xlabel='',
ylabel='Years')
sns.despine(); save('17_box_release_lag')
display(df.groupby('type', observed=True)[['release_year',
'years_to_netflix']].median().T)
```





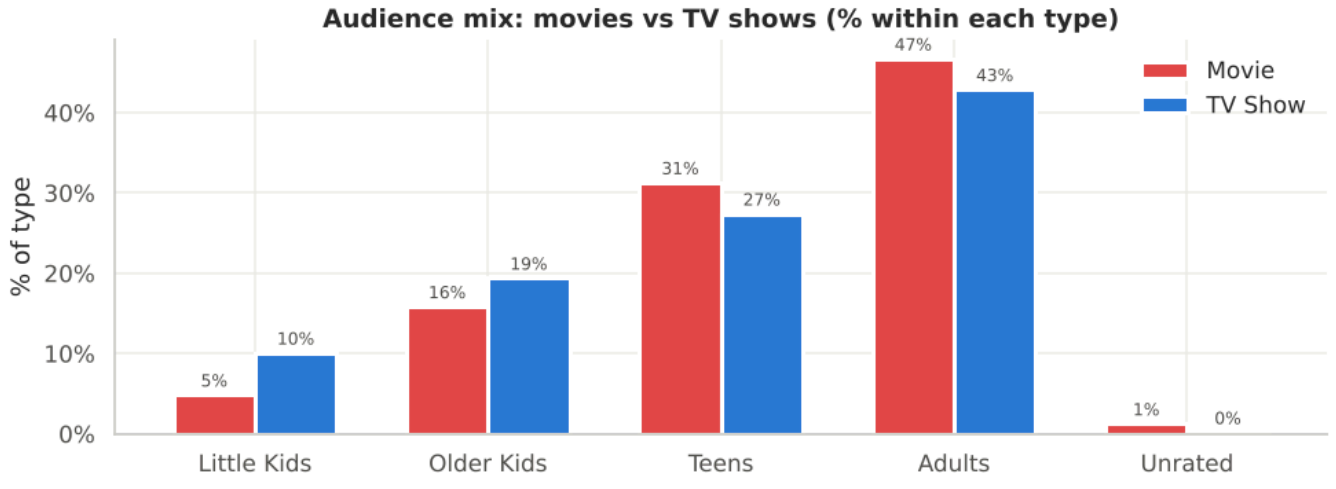
type	Movie	TV Show
release_year	2016.0	2018.0
years_to_netflix	2.0	0.0

Insights

- **TV shows are newer**: median release year 2018 vs 2016 for movies; the movie box is much taller, with outliers back to the 1940s.
- **TV shows reach Netflix in the same year (median 0 years)**, which is consistent with Netflix originals and simultaneous releases. **Movies arrive ~2 years after release** on median (after cinema and other windows), with many older catalogue films.
- Practical meaning: Netflix's TV slate is fresh and largely **self-produced**; its movie slate depends more on **licensing** older titles.

10.9 Ratings / audiences by content type

```
at = pd.crosstab(df['audience'], df['type'], normalize='columns') * 100
fig, ax = plt.subplots(figsize=(9, 3.4))
at.plot(kind='bar', color=[MOVIE, TV], width=0.7, edgecolor='white', linewidth=2,
ax=ax, rot=0)
label_bars(ax, fmt='{:.0f}%');
ax.yaxis.set_major_formatter(mtick.PercentFormatter(decimals=0))
ax.set(title='Audience mix: movies vs TV shows (% within each type)', xlabel='',
ylabel='% of type'); ax.legend(frameon=False, title='')
sns.despine(); save('18_audience_by_type')
```

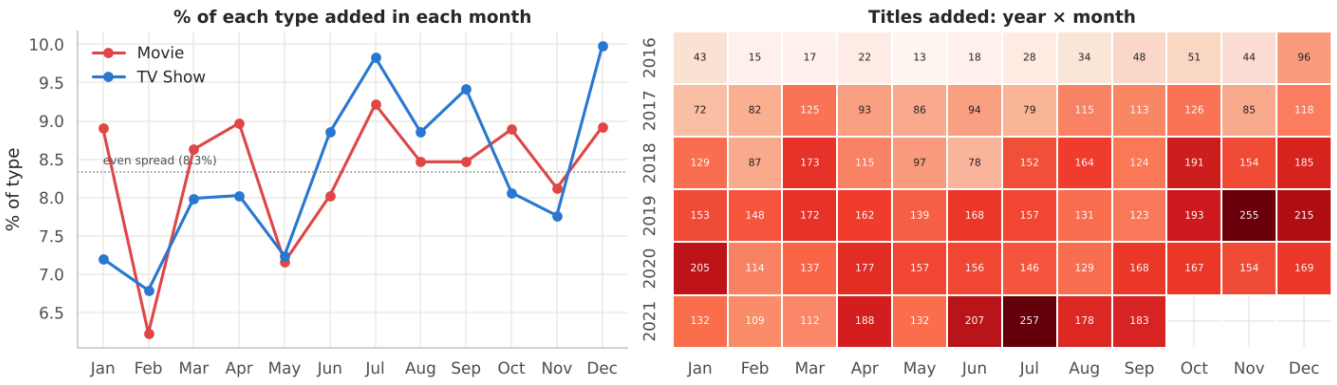


Insights

- Both formats are adult-heavy (47% of movies, 43% of TV), but **TV has twice the little-kids share (10% vs 5%)**, and Little + Older Kids ≈ **30% of TV vs 21% of movies**. Kids' content works better as series (repeat viewing, short episodes).
- **Teens (PG-13 / TV-14)** are ~31% of movies and ~27% of TV.

10.10 Best time to launch a TV show or movie (month × type, year × month heatmap)

```
mt = pd.crosstab(df['month_added'], df['type'])
mt_pct = mt.div(mt.sum()) * 100
mt_pct.index = pd.to_datetime(mt_pct.index.astype(int), format='%m').strftime('%b')
fig, axes = plt.subplots(1, 2, figsize=(13, 3.8), gridspec_kw={'width_ratios': [1,
1.2]})
for col in ['Movie', 'TV Show']:
    axes[0].plot(mt_pct.index, mt_pct[col], marker='o', lw=2, ms=6, color=PAL[col],
label=col)
axes[0].axhline(100/12, color=MUTED, ls=':', lw=1); axes[0].text(0, 100/12 + 0.1,
'even spread (8.3%)', fontsize=8, color='#52514e')
axes[0].set(title='% of each type added in each month', ylabel='% of type');
axes[0].legend(frameon=False)
ym = df[df.year_added.between(2016, 2021)]; ym = pd.crosstab(ym['year_added'],
ym['month_added'])
ym = ym.astype(float); ym.loc[2021, [10, 11, 12]] = np.nan   # data ends Sep 2021
ym.columns = pd.to_datetime(ym.columns.astype(int), format='%m').strftime('%b')
sns.heatmap(ym, cmap='Reds', annot=True, fmt='.0f', linewidths=1, linecolor='white',
annot_kws={'size': 7}, cbar=False, ax=axes[1])
axes[1].set(title='Titles added: year × month', xlabel='', ylabel='')
sns.despine(ax=axes[0]); save('19_best_launch_month')
```




```
tvw = pd.crosstab(df['weekday_added'], df['type'],
normalize='columns').mul(100).round(1)
display(tvw.T)
```

weekday_added	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
type							
Movie	10.2	13.9	14.8	17.2	25.5	9.1	9.3
TV Show	8.4	12.9	14.3	12.9	35.0	9.7	6.8

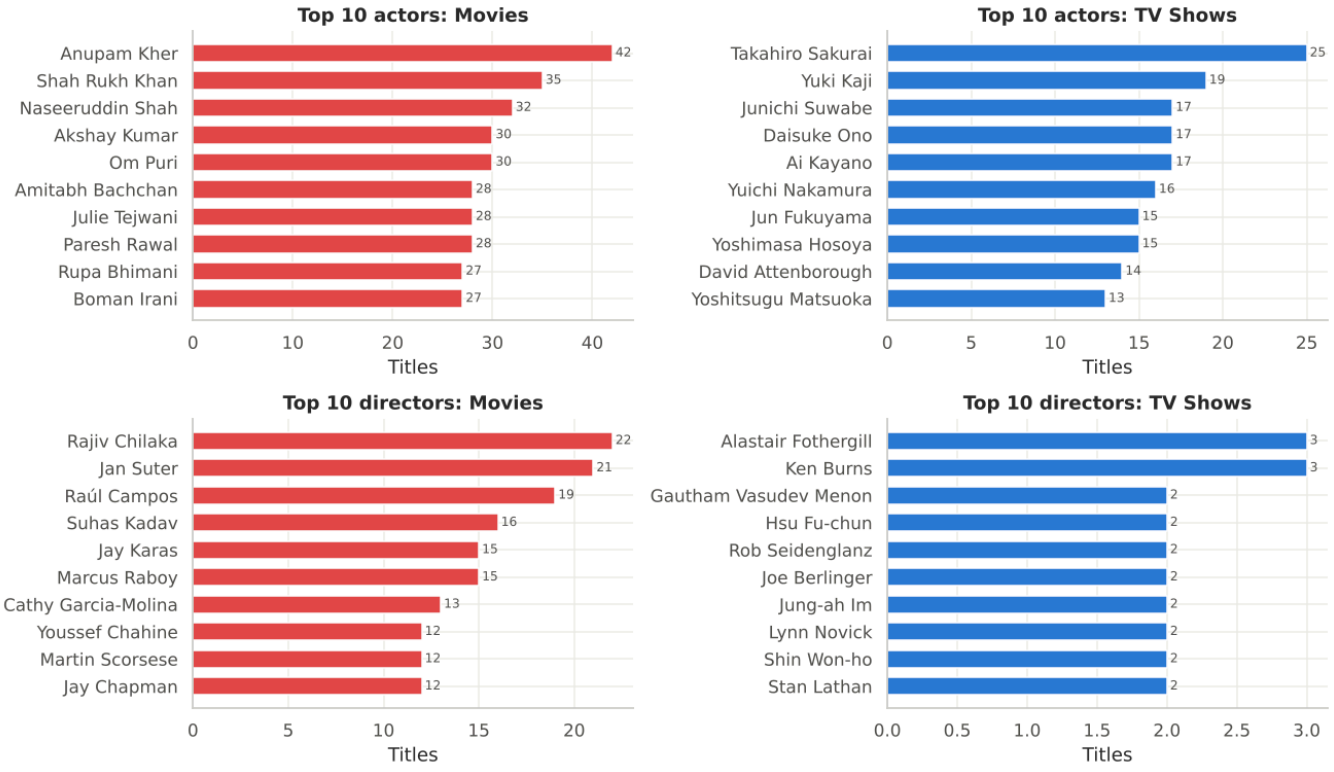
Insights: best time to launch

- **TV shows:** the highest shares are in **December (~10%) and July (~10%)**, followed by September; **February is the weakest (~7%)**. These match the **year-end holidays and summer vacations**, when people have time to binge.

- **Movies:** strongest in **July, April, December and January**; also weakest in February.
- **Weekday: 35% of TV shows and ~26% of movies are added on a Friday**, the single best day to set up a weekend binge.
- The heatmap shows **Q4 (Oct–Dec) was the hottest period in 2018 and 2019** (Nov 2019 = 255 titles), while in 2021 Netflix front-loaded the summer: **July 2021 (257) is the single biggest month on record**. Year-end and mid-summer are consistently strong.

10.11 Top actors and directors by content type

```
fig, axes = plt.subplots(2, 2, figsize=(12, 7))
for r_, (d, c) in enumerate([(known(df_cast, 'cast'), 'cast'), (known(df_director,
'director'), 'director')]):
    for k, (t_, col) in enumerate([('Movie', MOVIE), ('TV Show', TV)]):
        s = d.loc[d.type == t_, c].value_counts().head(10)
        ax = axes[r_, k]; ax.barh(s.index[::-1], s.values[::-1], color=col,
height=0.65); label_bars(ax)
        ax.set(title=f'Top 10 {"actors" if c == "cast" else "directors"}: {t_}s',
xlabel='Titles')
sns.despine(); save('20_top_actors_directors')
```

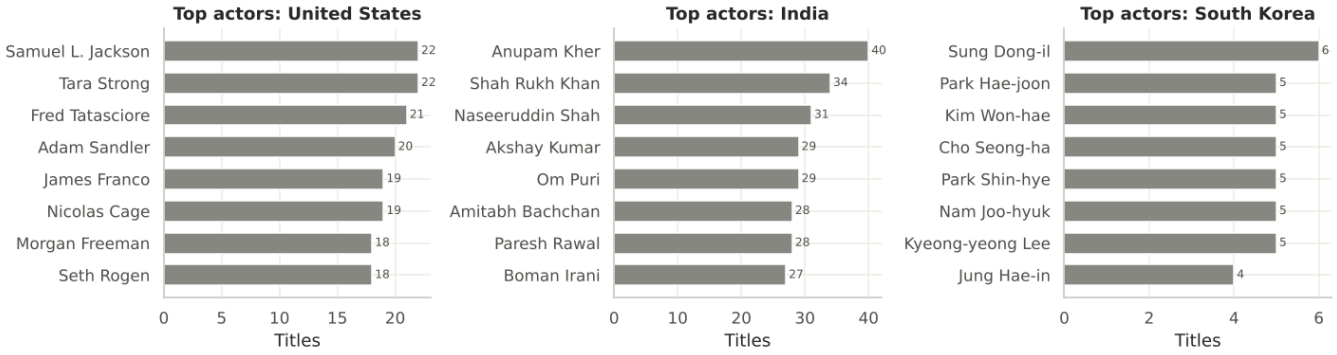


Insights

- **Movie actors are dominated by Indian stars:** Anupam Kher (42), Shah Rukh Khan (35), Naseeruddin Shah (32), Akshay Kumar, Om Puri, Paresh Rawal, Amitabh Bachchan. Two voice actors (Julie Tejwani, Rupa Bhimani) come from Indian kids' animation.
- **TV actors are dominated by Japanese voice actors** (Takahiro Sakurai 25, Yuki Kaji 19, Junichi Suwabe, Daisuke Ono, Ai Kayano), proof of anime's weight in the TV slate. **David Attenborough (14)** is the leading face of documentary series.
- **Top movie directors** include **Rajiv Chilaka (22, *Chhota Bheem* kids' animation)**, **Jan Suter (21) and Raúl Campos (19)**, who direct Latin-American stand-up specials, **Suhas Kadav (16)**, Marcus Raboy and Jay Karas (US stand-up), plus **Martin Scorsese (12)** for prestige cinema.

- **TV directors** rarely exceed 2–3 titles (and 91% are missing), so **TV is producer/creator-driven**, not director-driven.

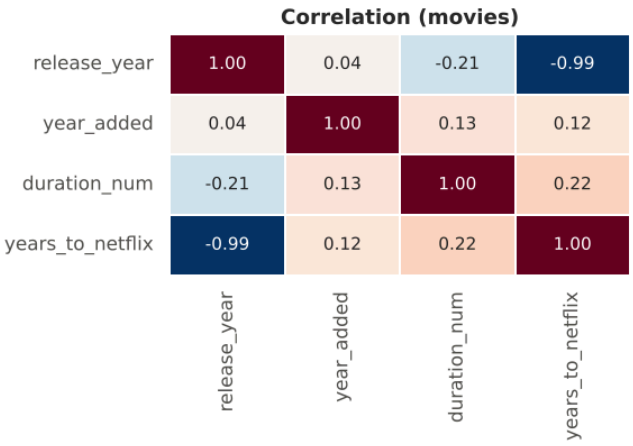
```
fig, axes = plt.subplots(1, 3, figsize=(13, 3.6))
for ax, c_ in zip(axes, ['United States', 'India', 'South Korea']):
    ids = df_country.loc[df_country.country == c_, 'show_id']
    s = known(df_cast, 'cast').loc[lambda d: d.show_id.isin(ids),
'cast'].value_counts().head(8)
    ax.barh(s.index[::-1], s.values[::-1], color=MUTED, height=0.65);
label_bars(ax); ax.set(title=f'Top actors: {c_}', xlabel='Titles')
sns.despine(); save('21_top_actors_by_country')
```

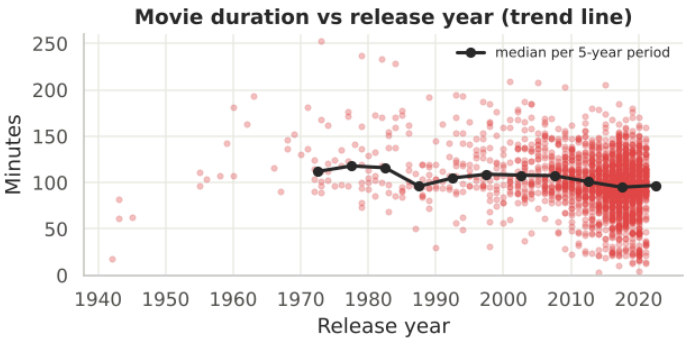


Insights: Every market has its own **local star system**: US titles are led by Hollywood names (Samuel L. Jackson 22, Adam Sandler 20, Nicolas Cage 19) and animation voice actors (Tara Strong 22, Fred Tatasciore 21); India by Bollywood veterans (Anupam Kher 40, Shah Rukh Khan 34); South Korea by K-drama leads (Sung Dong-il, Park Shin-hye), none of whom overlap. **Casting proven local stars is the fastest route to local relevance**; global names alone do not travel equally everywhere.

10.12 Relationship between numeric variables (correlation & pairwise view)

```
nm = movies[['release_year', 'year_added', 'duration_num',
'years_to_netflix']].astype(float).dropna()
fig, axes = plt.subplots(1, 2, figsize=(11, 3.8), gridspec_kw={'width_ratios': [1,
1.3]})
sns.heatmap(nm.corr(), annot=True, fmt='.2f', cmap='RdBu_r', vmin=-1, vmax=1,
center=0, linewidths=1, ax=axes[0], cbar=False)
axes[0].set(title='Correlation (movies)')
sns.scatterplot(data=nm.sample(2500, random_state=1), x='release_year',
y='duration_num', s=12, alpha=0.35, color=MOVIE, edgecolor=None, ax=axes[1])
trend = nm.groupby((nm.release_year // 5) * 5)['duration_num'].median().loc[1970:]
axes[1].plot(trend.index + 2.5, trend.values, color=INK, lw=2, marker='o', ms=5,
label='median per 5-year period'); axes[1].legend(frameon=False, fontsize=8)
axes[1].set(title='Movie duration vs release year (trend line)', xlabel='Release
year', ylabel='Minutes', ylim=(0, 260))
sns.despine(ax=axes[1]); save('22_correlation')
```





Insights

- `release_year` and `years_to_netflix` are almost perfectly negatively correlated **(−0.99)**, which is expected: older films have, by definition, waited longer to reach Netflix.
- **Movie duration has a weak negative relationship with release year (−0.21)**: the median runtime falls from ~112–118 min for 1970s–80s films to ~107 min in the 2000s and **~95–97 min since 2015**. Newer releases are getting **shorter**, matching streaming viewing habits.
- `year_added` is only weakly related to duration (0.13), so the length of what Netflix acquires has barely changed over time.

11. Outlier check (IQR method)

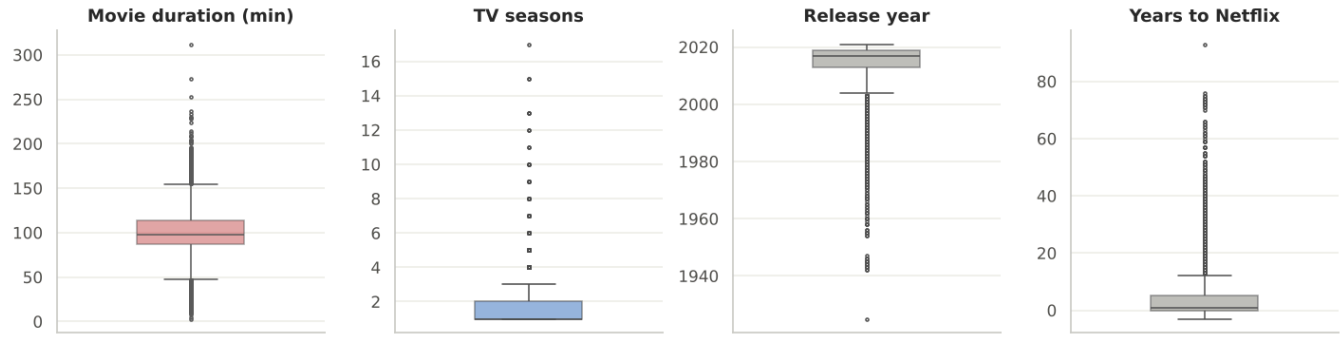
```
def iqr_report(s, name):
    q1, q3 = s.quantile([.25, .75]); iqr = q3 - q1; lo, hi = q1 - 1.5 * iqr, q3 +
1.5 * iqr
    out = s[(s < lo) | (s > hi)]
    return pd.Series({'Q1': q1, 'Q3': q3, 'IQR': iqr, 'lower fence': lo, 'upper
fence': hi,
                      'outliers': len(out), '% outliers': round(len(out) /
s.notna().sum() * 100, 1), 'min': s.min(), 'max': s.max()}, name=name)

rep = pd.concat([iqr_report(movies['duration_num'], 'Movie duration (min)'),
                 iqr_report(shows['duration_num'], 'TV seasons'),
                 iqr_report(df['release_year'].astype(float), 'Release year'),
                 iqr_report(df['years_to_netflix'].astype(float), 'Years to
Netflix')], axis=1).T
display(rep.round(1))
```

	Q1	Q3	IQR	lower fence	upper fence	outliers	% outliers	min	max
Movie duration (min)	87.0	114.0	27.0	46.5	154.5	450.0	7.3	3.0	312.0
TV seasons	1.0	2.0	1.0	-0.5	3.5	259.0	9.7	1.0	17.0
Release year	2013.0	2019.0	6.0	2004.0	2028.0	719.0	8.2	1925.0	2021.0
Years to Netflix	0.0	5.0	5.0	-7.5	12.5	972.0	11.0	-3.0	93.0

```
fig, axes = plt.subplots(1, 4, figsize=(13, 3.4))
for ax, (s, t_, col) in zip(axes, [(movies['duration_num'], 'Movie duration (min)',
MOVIE), (shows['duration_num'], 'TV seasons', TV),
                                   (df['release_year'], 'Release year', MUTED),
(df['years_to_netflix'].astype(float), 'Years to Netflix', MUTED)]):
    sns.boxplot(y=s, color=col, fliersize=2, width=0.4, ax=ax, boxprops={'alpha':
```

```
.55}); ax.set(title=t_, ylabel='')
sns.despine(); save('23_outliers_boxplots')
print('Longest movies:'); display(movies.nlargest(5, 'duration_num')[['title',
'country', 'release_year', 'duration']])
print('Shortest movies:'); display(movies.nsmallest(5, 'duration_num')[['title',
'listed_in', 'duration']])
print('Longest-running TV shows:'); display(shows.nlargest(5, 'duration_num')
[['title', 'country', 'duration']])
```



Longest movies:

	title	country	release_year	duration
4253	Black Mirror: Bandersnatch	United States	2018	312 min
717	Headspace: Unwind Your Mind	Unknown	2021	273 min
2491	The School of Mischief	Egypt	1973	253 min
2487	No Longer kids	Egypt	1979	237 min
2484	Lock Your Girls In	Unknown	1982	233 min

Shortest movies:

	title	listed_in	duration
3777	Silent	Children & Family Movies, Sci-Fi & Fantasy	3 min
2713	Sol Levante	Action & Adventure, Anime Features, Internatio...	5 min
1484	Cops and Robbers	Dramas	8 min
1557	Canvas	Children & Family Movies, Dramas	9 min
3535	American Factory: A Conversation with the Obamas	Documentaries	10 min

Longest-running TV shows:

	title	country	duration
548	Grey's Anatomy	United States	17 Seasons
2423	Supernatural	United States, Canada	15 Seasons
4798	NCIS	United States	15 Seasons
1354	Heartland	Canada	13 Seasons
4220	COMEDIANS of the world	United States	13 Seasons

Observations & treatment

- **Movie duration:** ~450 movies (7%) fall outside 46.5–154.5 min. Short ones are stand-up specials, kids' shorts and short documentaries; long ones are mostly Indian films and epics (*Black Mirror: Bandersnatch* at 312 min is interactive). **These are real titles, not errors**, so they are kept, and medians (not means) are reported.
- **TV seasons:** anything over 3 seasons is a statistical "outlier" because 67% of shows have one season. Long-running hits (e.g. *Grey's Anatomy*, 17 seasons) are valuable, not errors, so they are kept.

- **Release year:** with the middle 50% between 2013 and 2019, any title released **before 2004** is a statistical outlier (719 titles, 8%); kept because they represent the classic catalogue.
- **Years to Netflix:** negative values (14 rows) are genuine data inconsistencies and should be ignored in lag analysis; large positive values are classic films and are kept.
- **Conclusion:** no outlier removal is needed for this descriptive analysis; robust statistics (median, IQR) are used where skew matters.

Summary of EDA insights (distribution & relationships)

6.2 Distribution of variables & relationships between them

Variable	Distribution	Relationship observed
`type`	70% Movie / 30% TV	TV share of additions rising since 2018 (25% → 34%)
`release_year`	Left-skewed, peak 2017–18, median 2017	TV is newer (median 2018) than movies (2016)
`year_added`	Sharp rise 2016–19, plateau 2020–21	Movie additions falling, TV steady
Movie duration	Near-normal, centre 98 min, IQR 87–114	Rises with audience age (46 → 107 min); slightly shorter for newer films
TV seasons	Extremely right-skewed, 67% = 1 season	Very few renewals beyond 3 seasons
`rating` / audience	Adult-heavy (TV-MA 36%), little kids only 6.5%	TV has twice the little-kids share of movies
`country`	Long tail; US > India > UK	Asia = TV-heavy (JP, KR), India = movie-heavy
`genre`	International & Dramas lead	Reality, kids & romance growing; stand-up & docs shrinking
`month/weekday added`	Fairly flat by month; Friday spike	TV peaks in Jul/Dec; Feb lowest for both
`years_to_netflix`	Right-skewed, median 1	Strong negative correlation with release_year (−0.9)

12. Business insights

1. **Netflix is pivoting towards TV.** TV's share of new additions rose for three straight years (25% → 34%), and for 2021 releases TV shows outnumber movies (315 vs 277). Series drive repeat engagement and subscriber retention, and Netflix's own investment pattern confirms it.
2. **Growth has plateaued.** Additions peaked at 2,016 titles in 2019, then slowed in 2020–21 (COVID production halts, rising content costs). Future growth must come from **better targeting**, not just more titles.
3. **The catalogue is adult-first, with a family gap.** ~46% of titles are adult-rated and only 6.5% are for little kids, yet **Children & Family movies doubled (+100%) and Kids' TV grew +66% in 2018–20**. Netflix itself is moving into this gap.
4. **International content is the core of the brand.** "International Movies/TV Shows" are the #1 genre for both formats; 15% of titles are co-productions across countries. Mexico and Spain over-index on crime and thrillers; Canada on kids' animation.
5. **Each market has a distinct taste profile:**
    - *India:* movies (92%), dramas & comedies, Bollywood star-driven; additions fell 43% from the 2018 peak.
    - *Japan:* anime-led TV (63% TV); Japanese voice actors are the most frequent TV cast.
    - *South Korea:* K-drama/romantic TV (74% TV), additions nearly doubled 2018→2019.

- *UK:* British series and documentaries. *US:* the broadest mix, strongest in docs, comedy and kids.
- *Nigeria & Egypt:* went from almost zero to 42–54 titles/year within two years (Nigeria still growing in 2021).

6. **Short-lived TV is the norm.** Two-thirds of shows end after one season, which signals high experimentation (many pilots, few renewals) and limited series formats.
7. **Ideal movie length is clear.** Most movies are 87–114 min; kids' content is far shorter (46–68 min), and stand-up/docs fill the 60–90 min band cheaply.
8. **Release timing is deliberate.** Friday launches (28% overall, 35% of TV) and 1st-of-month licensing drops dominate; July and December are the peak months for TV, and February is the quietest.
9. **Original TV vs licensed movies.** TV arrives on Netflix in the same year it is released (median 0 years), consistent with originals, while movies arrive ~2 years later, consistent with licensing windows.
10. **Growing vs declining genres.** Reality TV (+95%), romantic movies (+60%) and kids' content are rising; stand-up (−46%) and documentaries (−29%) are shrinking.

13. Recommendations (simple action items)

1. **Make more TV series.** Keep raising the share of TV shows, especially **international dramas, crime series and kids' shows**. Series keep people subscribed month after month.
2. **Give good shows a second season.** Two out of three shows stop after season 1. Set clear rules for renewing shows that people finish, so popular stories can grow into long-running hits.
3. **Build more family and kids' content.** Content for little kids is only about 1 in 15 titles, yet kids and family is one of the fastest-growing areas. Add more animated series and family films, especially from India and Canada, where there is proven talent.
4. **Go local, country by country:**
   - **India:** keep making movies with popular local stars, and start testing **Indian web-series** to move beyond films.
   - **Japan:** invest in **anime** series that can travel worldwide.
   - **South Korea:** keep funding **K-dramas and romance** series; this market's output nearly doubled in a year.
   - **Mexico and Spain:** make more Spanish-language **crime and thriller series**, a proven strength there.
   - **Nigeria and Egypt:** the fastest-growing new markets. Partner with local studios to produce more local movies and series there.
   - **UK:** keep investing in British dramas and nature/documentary series.
5. **Keep movies around 90 to 110 minutes.** That is where most successful Netflix movies sit. Kids' movies can be shorter (around an hour).
6. **Launch big shows on a Friday, in July or December.** These are the months when most TV shows go live and people have holiday time to binge. Avoid February for big launches, or use it for smaller titles.
7. **Grow the right genres.** Put more money into **reality shows, romance, action and kids' content**, which are all growing. Spend carefully on stand-up specials and documentaries,

which Netflix is already adding less of.

8. **Co-produce to enter new markets.** 15% of titles are made by two or more countries. Partnering with local producers lowers cost and risk and speeds up entry into new countries.
9. **Refresh the classic movie library selectively.** Older films make up a small share; adding well-known classics can attract older audiences at low cost.
10. **Track what people watch, not just what is added.** Pair this catalogue view with viewing data so these bets can be measured and adjusted every quarter.

Prepared by Guntuku Hariprasad · Data: Netflix titles dataset (Scaler) · All numbers computed in this notebook.